

CAPABILITY-DRIVEN SELF-EVOLUTION OF AGENT MEMORY

Yaoqi Chen^{1,3,*} Yuru Feng^{2,3,*} Qianxi Zhang³ Baotong Lu³ Jianan Lu³
Zhirui Wang³ Shusen Xu³ Zewen Jin³ Zengzhong Li³
Cheng Li³ Qi Chen^{3,†}

¹ University of Science and Technology of China

² University of California, San Diego

³ Microsoft

* Equal contribution. † Corresponding author.

ABSTRACT

Memory self-evolution uses task feedback to iteratively improve executable memory programs that store and retrieve information from past interactions. Existing approaches typically adopt holistic evolution, deriving revision directions from mixed feedback and judging progress by overall performance. This can obscure optimization directions and hide capability-specific gains offset by regressions elsewhere, leaving promising directions underexplored. We introduce *capability-driven evolution*, which extends search guidance from overall performance to individual capability dimensions, preserving promising revisions and expanding exploration beyond the boundaries of holistic evolution. We propose PRISM, which uses dependency-aware capability selection to prioritize targets with potential cross-capability benefits and history-guided diagnosis to refine capability specialists. Trace-guided integration compares evaluated programs on paired differential cases, using their behavioral differences to consolidate complementary gains into a unified memory program. Experiments show that PRISM outperforms the strongest baselines by 10.54 and 7.83 percentage points on BEAM-1M and LongMemEval-M, respectively, demonstrating its effectiveness on million-token histories.

1 INTRODUCTION

Memory has become a key component of large language model (LLM) agents, enabling them to retain and leverage information from past interactions (Packer et al., 2023; Zhong et al., 2024; Kang et al., 2025; Salama et al., 2025). These interactions contain diverse information, including explicit facts, implicit preferences and dynamically changing states. Moreover, different requests call for diverse capabilities, such as factual retrieval, temporal tracking, preference recognition, and cross-session synthesis (Maharana et al., 2024; Wu et al., 2025; Tavakoli et al., 2026). Effective memory is therefore inherently multidimensional, requiring a memory program to support distinct yet interconnected capabilities across diverse tasks and evolving interactions (Xu et al., 2025; Rasmussen et al., 2025; Zhang et al., 2026b; Salama et al., 2025; Ye et al., 2026).

Self-evolving memory offers a promising solution by representing agent memory as an executable program, evaluating it on training tasks, and iteratively refining its implementation by an LLM based on task feedback (Pan et al., 2026; Liu et al., 2026b; Zhang et al., 2026a). A key challenge is identifying useful revision directions and determining which improvements are worth pursuing. Existing methods generally adopt *holistic evolution*, deriving optimization directions from mixed feedback across memory capabilities and judging progress mainly by overall task performance. Such compressed scalar guidance can confine the search to a limited region of capability space, with overall performance plateauing while some capability directions remain underexplored (Fig. 1(c)).

This limitation arises from two issues: **ambiguous optimization directions** and **hidden capability gains**. First, mixed feedback combines failures from tasks requiring different capabilities, which may suggest multiple possible revisions for the same memory component with different effects. For

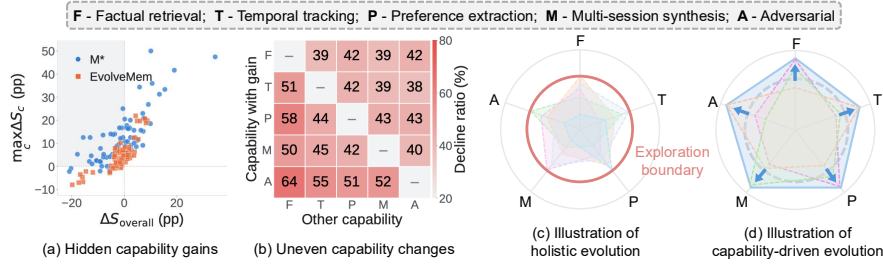


Figure 1: **Motivation for capability-driven evolution.** Analysis of M^* and EvolveMem evolution traces on BEAM. (a) Overall and maximum capability-level score changes for each revision. Many revisions improve at least one capability despite lower overall scores (upper-left region). (b) Among revisions that improve one capability (row), the percentage for which another capability’s score decreases (column). (c) Holistic evolution confines exploration to a limited region. (d) Capability-driven evolution enables broader exploration through explicit capability directions.

example, broader retrieval may improve multi-session reasoning but increase over-personalization by introducing irrelevant personal memories (Hu et al., 2026b; Pulipaka et al., 2026; Cao et al., 2026). Our analysis in Fig. 1(b) shows that such uneven effects are common. Before these directions are implemented and evaluated, their actual effects across capabilities remain unknown, making it difficult to determine which direction to pursue for overall improvement.

Second, a revision’s gains on one capability can be offset by regressions in others, leaving the overall score unchanged or even lower. Fig. 1(a) shows that 80.5% of revisions without an overall gain still improve at least one capability. This causes useful mechanisms to be overlooked before their benefits can be further refined, integrated, and reflected in overall performance. This issue is particularly pronounced near a performance plateau, where marginal gains are easily masked, further confining the search around the current solution.

Our key insight is to elevate the evolution space from a single overall-performance dimension to multiple capability-level dimensions. This motivates *capability-driven evolution*: Capability-specific feedback suggests focused diagnosis and guides explicit directions, while capability-level evaluation preserves promising revisions even without overall gains, enabling further refinement and broader exploration along individual capability dimensions (Fig. 1(d)). Crucially, this process also turns broader optimization directions into evaluated memory programs whose implementations and execution results reveal how differently these programs behave on the same tasks. These references provide guidance for combining useful mechanisms into a stronger unified memory program, rather than choosing among unexplored directions from mixed feedback.

We propose PRISM, which decomposes holistic memory evolution into capability-specific directions, analogous to a prism separating light. To achieve broader gains from each capability refinement step, *dependency-aware capability selection* exploits dependencies among capabilities to prioritize those whose improvement can also benefit others. Refinement starts from the best-performing program for the selected capability (specialist), allowing local gains to develop even when obscured by the overall score and expanding the search boundary. *History-guided diagnosis* then identifies informative failures that past programs handled well, while discounting repeatedly examined cases and capability combinations to broaden diagnostic coverage and uncover further revision opportunities. Finally, *trace-guided integration* consolidates capability specialists evolved along different directions. Using *paired differential cases*, it compares the base program with each specialist to expose their behavioral differences and guide how complementary gains should be incorporated. This translates broader capability-level exploration into a stronger unified memory program.

Our contributions are: (1) We introduce capability-driven evolution, a new paradigm for memory self-evolution that lifts search guidance to the capability dimension, expanding the exploration boundary. (2) We propose PRISM, which combines dependency-aware capability selection, history-guided diagnosis, and trace-guided integration to develop capability-level advances into a stronger memory program. (3) We evolve on shorter-context splits of BEAM and LongMemEval and test on their 1M-token versions, achieving improvements of 7.83–10.54 percentage points over the best-performing baselines and demonstrating effective generalization to million-token histories.

2 BACKGROUND AND RELATED WORK

Diverse Memory Capabilities. Agent memory enables agents to retain and reuse interaction history for reasoning, decision-making, and long-term interaction (Park et al., 2023; Packer et al., 2023). Diverse historical information and user needs impose heterogeneous requirements on memory systems, where design choices that benefit one capability may adversely affect others. For instance, personalized assistance requires leveraging a user’s preferences, while general information requests should remain unaffected by unrelated personal details (Zhong et al., 2024; Hu et al., 2026b). Appendix E provides more examples. These diverse requirements motivate different approaches to memory design, including *static* and *self-evolving* approaches.

Static Agent Memory. Static memory systems use predefined structures and fixed strategies for storing and retrieving information, with manually designed mechanisms for different memory capabilities. Mem0 (Chhikara et al., 2025) and SimpleMem (Liu et al., 2026a) emphasize preserving and consolidating salient information for *factual retrieval*. Temporal context receives particular attention in Zep (Rasmussen et al., 2025), Hindsight (Latimer et al., 2025) and TiMem (Li et al., 2026), supporting *temporal tracking* of events and changing states. MemoryBank (Zhong et al., 2024), MemInsight (Salama et al., 2025), MemoryOS (Kang et al., 2025), O-Mem (Wang et al., 2025), and EverMemOS (Hu et al., 2026a) draw on accumulated user information to support *preference extraction* and personalized responses. A-MEM (Xu et al., 2025), GAM (Yan et al., 2025), MRAgent (Ji et al., 2026), MemWeaver (Ye et al., 2026), and Amory (Zhou et al., 2026) emphasize connecting and combining information across interactions for *multi-session synthesis*.

Self-Evolving Agent Memory. Self-evolving memory methods use task feedback to iteratively refine memory structures and management strategies by an LLM, finally producing a fixed memory program. EvolveMem (Liu et al., 2026b) focuses on configuration-level optimization of retrieval strategies, while M* (Pan et al., 2026), MemEvolve (Zhang et al., 2026a) and MemPro (Liu et al., 2026c) pursue program-level evolution of full memory structures and processing logic. These methods rely on *holistic evolution* driven by overall scores. This mixed feedback can blur optimization directions, while aggregate scores can mask capability gains that are offset by regressions elsewhere, leading to stagnant evolution.

3 PRISMEM: CAPABILITY-DRIVEN MEMORY EVOLUTION

Problem Formulation. PRISMEM aims to improve how an agent stores and retrieves conversational history by optimizing its executable memory program P . Let $x = (\mathcal{H}, q, y, E)$ denote a task instance with history $\mathcal{H}$, question q , reference answer y , and reference evidence E . The program constructs memory from $\mathcal{H}$ and retrieves content for answering q , receiving a task score $s(P; x) \in [0, 1]$. We denote the average score of P over a task set $\mathcal{S}$ as $s(P; \mathcal{S})$. The task set is split into $\mathcal{S}_{\text{train}}$, $\mathcal{S}_{\text{valid}}$, and $\mathcal{S}_{\text{test}}$. Memory self-evolution iteratively evaluates P on $\mathcal{S}_{\text{train}}$, leverages an LLM to diagnose problems and refine P , and pursues improvements on $\mathcal{S}_{\text{valid}}$. After reaching maximum iterations, the best-performing program is frozen for evaluation on $\mathcal{S}_{\text{test}}$.

Fine-grained Memory Interface. The memory interface determines the scope within which the memory program can be modified during self-evolution. We decompose the memory program into five components: *Extraction*, *Indexing*, *Planning*, *Retrieval*, and *Answer*. Compared with the commonly used “add” and “retrieve” abstraction in agent memory systems, this decomposition provides more localized modification boundaries, helping the coding model identify which stage is responsible for a failure and revise the corresponding component more precisely.

Capability Set for Our Study. PRISMEM uses capabilities to organize evolution and guide program refinement. In our experiments, we instantiate the framework with five representative agent memory capabilities, denoted by $\mathcal{C}$ (Table 1). We select these capabilities through a broad review of widely used memory benchmarks (Maharana et al., 2024; Wu et al., 2025; Tavakoli et al., 2026) and existing memory systems (Zhong et al., 2024; Rasmussen et al., 2025; Chhikara et al., 2025; Xu

Table 1: Five memory capabilities used in our experiments.

Capability	Memory requirement
Factual retrieval (F)	Preserve and retrieve explicit facts, values, and other localized details.
Temporal tracking (T)	Order events, determine recency, track states and knowledge updates.
Preference extraction (P)	Recover and apply user preferences, instructions and constraints.
Multi-session synthesis (M)	Aggregate and synthesize evidence across multiple sessions or events.
Adversarial (A)	Abstain when memory lacks supporting evidence, or cautiously identify and resolve contradictory or misleading evidence.

et al., 2025), focusing on recurring memory requirements across these settings ¹. Each capability captures a distinct and essential memory requirement.

We offline annotate tasks in $\mathcal{S}_{\text{train}}$ and $\mathcal{S}_{\text{valid}}$ using an LLM to assign each task one primary capability $\ell(x) \in \mathcal{C}$ and up to two secondary capabilities $\sigma(x) \subseteq \mathcal{C} \setminus \{\ell(x)\}$. These capability labels organize feedback and guide evolution but are never provided to the memory program during evaluation, ensuring a fair comparison with baselines. Let $\mathcal{S}^c = \{x \in \mathcal{S} : \ell(x) = c\}$ denote the subset of tasks associated with primary capability c .

3.1 CAPABILITY-DRIVEN EVOLUTION

PRISMEM organizes evolution into three stages to expand the search boundary along individual capability dimensions. First, the cold start stage addresses broad weaknesses in the seed program. Second, the capability refinement stage performs *dependency-aware capability selection* to prioritize capabilities and *history-guided diagnosis* to provide focused revisions, while capability-level evaluation preserves promising variations. Finally, *trace-guided integration* consolidates these specialized programs into a stronger, unified memory program. Figure 2 illustrates the evolution process, and Algorithm 1 in Appendix presents the pseudocode.

3.1.1 STAGE I: COLD START

The initial seed memory program P_0 is deliberately simple, providing a broad search space but potentially leaving basic processing weaknesses across multiple capabilities. Cold start jointly considers capability-wise feedback across all capabilities over $T_1 = 5$ rounds to quickly establish a balanced base program for subsequent capability refinement. The program with the highest validation score becomes the starting point for the subsequent stages, while the evolution history is retained to guide later capability selection and refinement.

To guide these early revisions, we construct a metric table that aggregates evidence-based diagnostics by capability, preserving capability-specific differences and reducing diagnostic ambiguity. The diagnostics measure semantic similarity between reference evidence and (i) extracted memory units, (ii) content retrieved using the evidence as a query cue, and (iii) content retrieved for the actual question. These metrics assess information preservation during extraction, accessibility through indexing, and retrieval effectiveness. Comparing these capability-by-component patterns helps the model localize broad processing weaknesses and prioritize concrete revisions in early evolution.

3.1.2 STAGE II: CAPABILITY REFINEMENT

Next, we refine individual capabilities of the memory program obtained from cold start stage, retaining valuable capability-specific breakthroughs when they are obscured by overall performance. Let $\mathcal{P}_t$ denote the set of candidate programs retained up to iteration t . For each capability c , we track its best-performing program as the specialist $P_t^*(c)$, and denote the globally best program on the complete task set by G_t . When a target capability c_t is selected, refinement is initialized from its specialist, i.e., $P_t = P_t^*(c_t)$. Each retained candidate is evaluated on tasks across all capabilities, so a single program can simultaneously serve as the specialist for multiple capabilities.

¹Table 5 in Appendix illustrates how different question types from some benchmarks relate to the chosen capabilities. Hereafter, we use **F**, **T**, **P**, **M**, and **A** as shorthand for the full capability names.

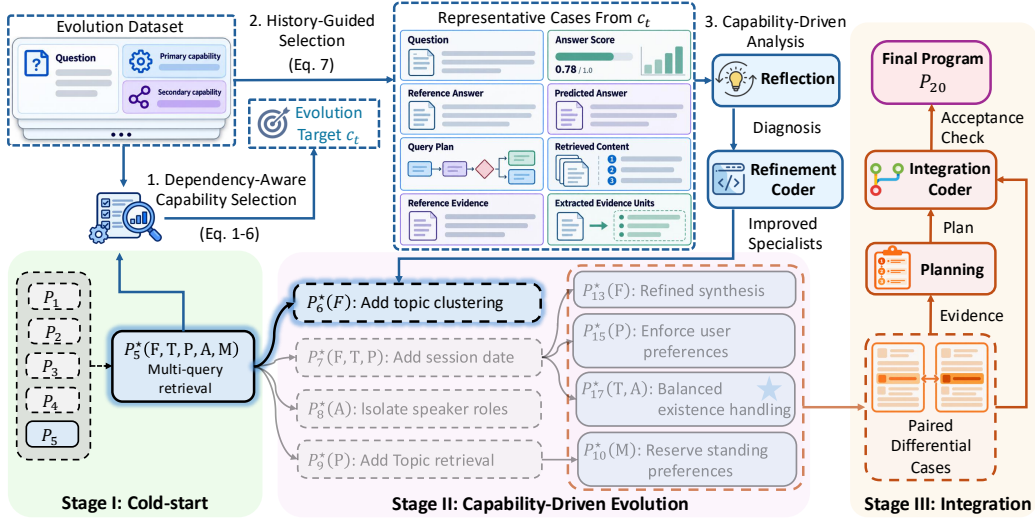


Figure 2: **Overview of PRISMEM.** *Stage I: Cold start* quickly establishes a balanced program and initial evolution history. *Stage II: Capability refinement* uses dependency-aware capability selection and history-guided case selection to target capabilities and diagnose informative failures, respectively, refining their specialists through reflection and code revision. *Stage III: Trace-guided integration* compares the base program ($\star$) and other capability specialists on paired differential cases, using their behavioral differences to guide integration into a unified memory program.

Dependency-Aware Capability Selection. Selecting a capability for the next refinement is important because it determines how effectively the limited refinement budget expands the capability boundary. We prioritize capabilities that are weak themselves or appear to be bottlenecks for other capabilities, while downweighting those that have already received substantial refinement budget. A capability may exhibit not only a *direct deficit* on tasks where it is the primary requirement, but also *latent error pressure* on tasks where it appears as a secondary requirement. For example, a deficiency in temporal tracking may affect tasks primarily targeting multi-session synthesis when they also require temporal information. To quantify this latent pressure, we consider tasks with primary capability c_p and secondary capability c_s . Since such co-occurrence does not necessarily imply that deficiencies in c_s affect c_p , we first define a structural base-rate prior:

$$D(c_p \rightarrow c_s) = \frac{\sum_{x \in \mathcal{S}_{\text{train}}^{c_p}} \mathbf{1}[c_s \in \sigma(x)]}{|\mathcal{S}_{\text{train}}^{c_p}|}, \quad (1)$$

which measures how often primary- c_p tasks also require c_s . At iteration t , let

$$e_{t,c_s}(x) = 1 - s(P_t^*(c_s); x), \quad \bar{e}_{t,c_s}(c_p) = \frac{1}{|\mathcal{S}_{\text{train}}^{c_p}|} \sum_{x \in \mathcal{S}_{\text{train}}^{c_p}} e_{t,c_s}(x) \quad (2)$$

denote the instance-level and average error of the c_s specialist on primary- c_p tasks. We then measure how disproportionately this error is concentrated on tasks that require c_s :

$$F_{t,c_s}(c_p \rightarrow c_s) = \frac{\sum_{x \in \mathcal{S}_{\text{train}}^{c_p}} e_{t,c_s}(x) \mathbf{1}[c_s \in \sigma(x)]}{\sum_{x \in \mathcal{S}_{\text{train}}^{c_p}} e_{t,c_s}(x)}, \quad (3)$$

with $F_{t,c_s}(c_p \rightarrow c_s) = 0$ when the denominator vanishes. Intuitively, if c_s is not a bottleneck for c_p , its error should be distributed roughly according to the base rate, i.e., $F \approx D$. Thus, an excess gap $F - D > 0$ indicates that deficiencies in c_s are disproportionately associated with errors on c_p . Scaling this gap by the primary task error yields the incoming error pressure:

$$I_t(c_p \rightarrow c_s) = \bar{e}_{t,c_s}(c_p) [F_{t,c_s}(c_p \rightarrow c_s) - D(c_p \rightarrow c_s)]_+, \quad (4)$$

where $[z]_+ = \max(z, 0)$. For a candidate capability c , we combine its direct deficiency with its strongest cross-capability impact:

$$U_t(c) = \bar{e}_{t,c}(c) + \max_{c_p \in \mathcal{C} \setminus \{c\}} I_t(c_p \rightarrow c). \quad (5)$$

Crucially, using the maximum rather than a sum avoids degree-centrality bias and focuses the scheduler on the most acute structural bottleneck. Finally, to avoid repeatedly allocating the search budget to plateaued capabilities, we discount urgency by the historical allocation count $n_t(c)$:

$$\rho_t(c) = \frac{U_t(c)}{1 + n_t(c)}, \quad c_t = \arg \max_{c \in \mathcal{C}} \rho_t(c). \quad (6)$$

Thus, a capability c_t is prioritized either for its direct deficiency or for its strongest upstream impact on other capabilities, while its priority naturally decreases with repeated exploration.

History-Guided Diagnosis. In later-stage evolution, the memory program is relatively mature, so further improvements require fine-grained diagnosis of individual failures. Inspecting failure cases through their execution results helps pinpoint whether an error arises from noisy extraction, defective memory representation, retrieval failure, or answer misinterpretation, translating behavioral symptoms into actionable code revisions. However, diagnostic cases can be lengthy. We therefore select $K = 5$ cases that are severe, historically actionable, and collectively diverse, allowing each refinement step to explore a broader and more valuable set of optimization directions.

Specifically, we first consider training task instances whose primary capability is the selected target c_t and filter out those with scores above $\delta = 0.8$, retaining cases that are not fully correct. We then rank the remaining failures by a history-guided priority weight:

$$w_t(i) = \frac{1 + (1/|\mathcal{P}_t(c_t)|) \sum_{P \in \mathcal{P}_t(c_t)} s(P; x_i)}{(1 + m_t(i))(1 + m_t(\sigma(x_i)))}. \quad (7)$$

Here, $\mathcal{P}_t(c_t)$ contains valid programs produced in previous iterations that also targeted c_t . The numerator measures historical performance; a high historical score for a current failure identifies a regression witness, whose contrast with the current failure helps localize the effects of recent revisions. The denominator discounts cases that have been selected frequently ($m_t(i)$) and secondary-capability combinations that have been repeatedly covered ($m_t(\sigma(x_i))$), promoting diverse coverage and reducing redundant optimization of similar failures. This enables the limited refinement budget to explore broader and more informative optimization directions.

3.1.3 STAGE III: TRACE-GUIDED INTEGRATION

While capability specialists extend the performance boundary along different dimensions, their improvements may remain isolated. The integration stage consolidates these distributed gains into a unified memory program, starting from the overall best program so far and progressively incorporating capability specialists from other directions. A planning model determines the integration order based on the lineage topology, processing specialists from those closest to the base program to those farther away, and produces a step-by-step integration plan. A coding model then executes the plan.

This integration is enabled by the evaluation results of retained programs. Since all programs are evaluated on the same training task set, we can compare revisions on the same cases and trace their effects through the evolution lineage, revealing their capability-specific gains and side effects. We therefore introduce a *paired differential cases* strategy to guide integration. For each capability, we select the training case with the largest score gap between the base program and the specialist being integrated. Comparing their execution results exposes the specialist’s behavioral differences, helping the planning model determine how its gains and side effects should be incorporated.

The resulting plan is executed for up to $T_3 = 3$ rounds. If integration fails to improve the overall score or causes any capability to decrease by more than θ , the failed implementation and its execution results on the same differential cases are returned to the coding model for another integration. Overall, this process aims to translate complementary capability-specific improvements into stronger overall performance while mitigating regressions across capabilities.

3.2 EFFICIENT EVOLUTION

Self-evolution repeatedly revises and evaluates memory programs, incurring substantial inference costs. We reduce this cost through *compact diagnostic contexts* and *revision-aware execution reuse*. First, diagnostic cases provide the information needed to identify program failures, including the

question, reference answer and evidence, retrieved content, and model prediction. Full traces can be lengthy, so we construct compact diagnostic contexts by removing redundant content and marking omissions while retaining snippets relevant to the question and answer. This reduces diagnostic-case token cost by 44% on average while preserving key information for diagnosis.

Second, evaluating each candidate program requires executing the memory pipeline, but revisions may affect only a subset of its components. Using the fine-grained memory interface, we compare each candidate with its parent and reuse outputs from unaffected components. For example, a retrieval-only revision can reuse the constructed memory and skip extraction and indexing. Selectively reusing completed results reduces redundant processing and accelerates program evaluation.

4 EXPERIMENTS

We evaluate PRISMEM on four questions: **Q1:** How does PRISMEM perform compared with existing methods? **Q2:** What is the evolution cost of PRISMEM? **Q3:** How much does each component contribute to PRISMEM’s performance? **Q4:** Can PRISMEM expand the capability boundary and integrate the resulting improvements into the final program?

4.1 EXPERIMENTAL SETUP

Benchmarks. We evaluate on two widely used agent memory benchmarks, BEAM (Tavakoli et al., 2026) and LongMemEval (Wu et al., 2025), which cover diverse question types over long, multi-session histories. All evolution methods train their memory programs for 20 iterations on the smaller splits, BEAM-100K and LongMemEval-S, and are then tested on the longer splits with different content, BEAM-1M and LongMemEval-M, with over 1 million tokens. Dataset details can be found in Appendix B.1. This protocol tests whether the learned design choices generalize to longer contexts.

Baselines. We compare PRISMEM with seven representative agent memory methods, including static and self-evolving approaches. Static baselines include Mem0 (Chhikara et al., 2025) and A-MEM (Xu et al., 2025), which organize and update structured memories; HippoRAG2 (Gutiérrez et al., 2025), which supports memory-enhanced retrieval with graphs; and SimpleMem (Liu et al., 2026a) and LightMem (Fang et al., 2026), which emphasize efficient memory construction and retrieval. Self-evolving methods include M* (Pan et al., 2026), which evolves executable memory programs through population-based reflective code search, and EvolveMem (Liu et al., 2026b), which uses failure diagnosis to adapt retrieval infrastructure and answer-generation configurations. Both adopt holistic evolution, using aggregate task performance as the primary signal for exploration.

Models. We use Qwen3.8-27B (Qwen, 2026) as the task, reflection, coding, and judging model, and Qwen3-Embedding-8B (Qwen, 2025) as the embedding model. Inference runs on NVIDIA A100 GPUs (NVIDIA, 2020) with vLLM 0.20.0 (Kwon et al., 2023) under Python 3.12. We also report results with GPT-5.5 (OpenAI, 2026) on BEAM only, due to its higher cost (Appendix C.1).

Metrics. Task performance is measured by the LLM judge scores provided by each benchmark. Overall score is averaged over all questions, while capability-level scores are averaged over questions grouped by the native question-type mapping in Table 5. Inference cost is measured by the average number of task LLM input and output tokens per question. For self-evolving methods, we additionally report the token consumption of the whole evolution procedure. Each experiment is run three times, and results are reported as mean $\pm$ standard deviation.

4.2 OVERALL PERFORMANCE (Q1)

Table 2 shows the main results on two million-token memory benchmarks. Static methods show different capability strengths: HippoRAG2 leads in factual retrieval, while A-MEM is competitive in preference extraction. Holistic evolution yields only limited gains over static methods, with M* improving over the strongest static baseline by 0.99 pp on BEAM and EvolveMem by 0.34 pp on LongMemEval. This limited improvement reflects how mixed feedback can obscure optimization directions and aggregate scores can mask capability gains offset by regressions elsewhere.

Table 2: Performance on BEAM-1M and LongMemEval-M with Qwen3.8-27B. “Score” denotes the benchmark-provided LLM judge score, and “Avg. #Tokens” denotes the task LLM’s average token usage per question. All results are reported as mean $\pm$ std. Best results are in **bold**.

Method	Score (%) $\uparrow$						Avg. #Tokens (K) $\downarrow$
	Factual retrieval	Temporal tracking	Preference extraction	Multi-session synthesis	Adversarial	<i>Overall</i>	
<i>BEAM-1M</i>							
Mem0	54.74 $\pm$ 3.06	46.51 $\pm$ 0.79	60.48 $\pm$ 2.39	38.02 $\pm$ 0.63	49.11 $\pm$ 1.24	48.95 $\pm$ 0.61	190.28 $\pm$ 0.20
A-MEM	56.21 $\pm$ 1.83	46.22 $\pm$ 0.62	65.39 $\pm$ 0.52	53.87 $\pm$ 0.86	41.13 $\pm$ 0.51	51.56 $\pm$ 0.62	428.95 $\pm$ 3.58
HippoRAG2	65.24 $\pm$ 1.26	47.06 $\pm$ 1.85	66.43 $\pm$ 1.57	44.83 $\pm$ 0.60	44.82 $\pm$ 2.28	51.86 $\pm$ 0.87	207.70 $\pm$ 0.19
SimpleMem	48.55 $\pm$ 3.33	36.82 $\pm$ 0.47	48.85 $\pm$ 1.19	21.87 $\pm$ 2.00	50.68 $\pm$ 0.40	40.18 $\pm$ 0.76	95.72 $\pm$ 0.24
LightMem	46.66 $\pm$ 0.82	33.40 $\pm$ 0.65	36.64 $\pm$ 0.63	16.70 $\pm$ 0.89	55.45 $\pm$ 0.94	36.44 $\pm$ 0.38	86.20$\pm$ 0.05
M*	61.29 $\pm$ 5.14	48.98 $\pm$ 5.87	71.91 $\pm$ 3.40	42.19 $\pm$ 4.88	46.02 $\pm$ 7.52	52.85 $\pm$ 2.49	89.47 $\pm$ 5.11
EvolveMem	41.80 $\pm$ 4.77	39.74 $\pm$ 3.15	52.21 $\pm$ 1.53	21.67 $\pm$ 3.07	45.67 $\pm$ 3.50	40.01 $\pm$ 1.46	98.40 $\pm$ 4.67
PRISMEM	74.17$\pm$ 4.87	57.26$\pm$ 2.15	75.58$\pm$ 3.91	54.12$\pm$ 3.37	64.32$\pm$ 6.51	63.39$\pm$ 1.02	100.59 $\pm$ 5.02
<i>LongMemEval-M</i>							
Mem0	69.79 $\pm$ 1.80	63.54 $\pm$ 0.90	43.33 $\pm$ 0.00	63.54 $\pm$ 1.80	63.33 $\pm$ 5.77	62.50 $\pm$ 0.50	5772.67 $\pm$ 4.12
A-MEM	87.50 $\pm$ 1.56	59.90 $\pm$ 0.90	44.44 $\pm$ 1.92	54.17 $\pm$ 1.80	96.67$\pm$ 5.77	67.33 $\pm$ 0.76	8873.92 $\pm$ 14.45
HippoRAG2	89.58$\pm$ 1.80	47.92 $\pm$ 0.90	24.44 $\pm$ 1.92	40.63 $\pm$ 3.13	90.00 $\pm$ 10.00	58.67 $\pm$ 1.26	6185.48 $\pm$ 5.01
SimpleMem	82.81 $\pm$ 1.56	74.48 $\pm$ 2.39	11.11 $\pm$ 1.92	67.71 $\pm$ 1.80	73.33 $\pm$ 11.55	66.50 $\pm$ 0.50	2624.68 $\pm$ 7.90
LightMem	47.92 $\pm$ 0.90	74.48 $\pm$ 0.90	24.44 $\pm$ 1.92	58.33 $\pm$ 1.80	96.67$\pm$ 5.77	57.00 $\pm$ 0.87	1888.30 $\pm$ 1.28
M*	76.04 $\pm$ 5.49	47.40 $\pm$ 3.93	12.22 $\pm$ 5.09	67.71 $\pm$ 4.77	93.33 $\pm$ 11.55	56.83 $\pm$ 3.21	1528.94$\pm$ 30.34
EvolveMem	78.13 $\pm$ 3.13	69.79 $\pm$ 2.39	42.22 $\pm$ 3.85	65.63 $\pm$ 3.13	70.00 $\pm$ 20.00	67.67 $\pm$ 1.53	2749.94 $\pm$ 27.18
PRISMEM	82.29 $\pm$ 3.93	75.00$\pm$ 3.13	47.78$\pm$ 1.92	82.29$\pm$ 3.61	96.67$\pm$ 5.77	75.50$\pm$ 1.80	2232.65 $\pm$ 32.72

In contrast, PRISMEM achieves the best performance in nearly all capability comparisons, improving overall scores over the strongest static and self-evolving baselines by 8.17–11.53 pp and 7.83–10.54 pp, respectively. These results demonstrate the effectiveness of capability-driven evolution: capability refinement expands exploration along individual dimensions and develops useful revisions beyond holistic evolution, while trace-guided integration consolidates these gains into a stronger program. These gains are achieved with token usage comparable to that of other methods.

PRISMEM also generalizes well across context lengths and underlying models. Programs evolved on BEAM-100K and LongMemEval-S retain their advantages on the longer BEAM-1M and LongMemEval-M splits. With GPT-5.5, PRISMEM also outperforms the baselines by 6.17–20.41 pp on BEAM-1M (Appendix C.1), showing that capability-driven evolution can produce effective memory programs across different underlying models. Additional cross-model and cross-dataset transfer experiments further support the transferability of the evolved programs (Appendix C.2).

4.3 EVOLUTION EFFICIENCY (Q2)

We compare the token consumption across self-evolving methods under the same 20-round evolution budget. The results are summarized in Table 3. LongMemEval incurs higher evolution costs because each question has an independent history, resulting in higher per-round extraction costs than BEAM. Overall, EvolveMem consumes fewer tokens because it evolves only the retrieval and answer configurations, whereas M* and PRISMEM evolve the full memory program and incur additional extraction costs. Despite this, PRISMEM reduces evolution cost by 4%–30% over M*, enabled by compact diagnostic contexts and revision-aware execution reuse introduced in Section 3.2.

Table 3: Evolution tokens (millions, $\downarrow$) with Qwen3.8-27B on two benchmarks.

Method	BEAM	LongMemEval
M*	76.5 $\pm$ 7.1	202.3 $\pm$ 10.3
EvolveMem	69.2 $\pm$ 7.3	102.7 $\pm$ 8.4
PRISMEM	73.7 $\pm$ 6.4	143.3 $\pm$ 9.6



Figure 3: **Evolution trace analysis.** (a) Gains of each Stage II iteration relative to program 05 obtained after cold start. Cell numbers are the creation iteration of the retained specialist for each capability; dots mark specialist updates. Scores improve intermittently across capabilities as refinement progresses. (b) Capability-wise scores of M^* , EvolveMem, and PRISM. Solid lines show the final selected programs, while dashed lines show the best capability-wise score reached across all iterations. PRISM expands the performance boundary, while its final integrated program closely matches its historical capability-wise best.

4.4 ABLATION STUDY (Q3)

To examine the contribution of each design choice, we compare four variants of PRISM on BEAM with Qwen3.8-27B. The results are shown in Table 4. Removing capability-driven evolution falls back to conventional holistic evolution and reduces the score by 10.15 pp. Without capability-level optimization, mixed feedback can blur optimization directions, while useful capability-specific revisions may not receive further refinement when they do not improve overall performance. This large drop highlights capability-driven evolution as a primary source of improvement.

Removing trace-guided integration reduces the score by 3.16 pp, but the variant still outperforms the baselines. This shows that optimizing along explicit capability directions already improves evolution by reducing mixed feedback and focusing revisions, while integration guided by evaluated programs and paired differential cases across capabilities provides additional gains in overall performance.

Using a fixed round-robin schedule instead of dependency-aware capability selection reduces the score by 2.55 pp, supporting targeted refinement of weak capabilities and potential bottlenecks for multi-capability tasks. Random case selection reduces the score by 5.26 pp, highlighting the importance of matching diagnostic cases to each stage’s objective. Our case selection provides capability-relevant failures for focused refinement and contrasting execution results that expose useful revisions and potential side effects during integration.

Table 4: Ablations with Qwen3.8-27B on BEAM-1M. Δ is relative to full PRISM.

Variant	Overall score (%) $\uparrow$	Δ (pp)
Full PRISM	63.39 $\pm$ 1.02	—
w/o capability evolution	53.24 $\pm$ 2.38	−10.15
w/o trace-guided integration	60.23 $\pm$ 1.86	−3.16
Round-robin target selection	60.84 $\pm$ 1.13	−2.55
Random case selection	58.13 $\pm$ 1.25	−5.26

4.5 EVOLUTION TRACE ANALYSIS (Q4)

We analyze PRISM’s evolution trajectories on BEAM with Qwen3.8-27B. In Figure 3a, capability scores improve intermittently throughout Stage II, showing refinements progressively expand performance along each capability dimension. Refining one capability can also update specialists for others, reflecting dependencies among capabilities and our dependency-aware selection of targets that can benefit multiple capabilities simultaneously. Figure 3b shows that capability-driven evolution expands beyond the performance boundary of holistic evolution, uncovering better optimization directions. Trace-guided integration then successfully consolidates these gains, with the final program closely matching the peak capability-wise performance reached during evolution.

5 CONCLUSION

We introduced capability-driven evolution to expand memory program optimization along explicit capability dimensions. Capability-specific feedback clarifies revision directions, while capability-level evaluation preserves gains obscured by overall scores. PRISMEM combines dependency-aware capability selection and history-guided diagnosis to refine capability specialists, then uses trace-guided integration to consolidate their complementary gains into a unified memory program. On BEAM and LongMemEval, PRISMEM outperforms the strongest baselines by 7.83–10.54 pp on million-token histories. We hope this work encourages further exploration of capability-driven evolution for building stronger agent memory.

AI USE STATEMENT

Generative AI tools assisted with manuscript drafting and language editing. LLMs were also used to evolve and evaluate memory programs for LLM agents. The authors reviewed all AI-assisted content and take responsibility for the final manuscript.

REPRODUCIBILITY STATEMENT

We describe the models and evaluation protocol in Section 4.1, and provide implementation details and evolution pseudocode in Appendix A. Dataset splits and evolution parameters are detailed in Appendix B. The evolution prompt templates and source code of the final evolved memory programs are included in Appendices F and G, respectively.

REFERENCES

- Ruike Cao, Fanyu Zhao, Fugen Yao, Liang Dong, Jian Xu, Guanjin Jiang, Yifei Zhao, Han Zhang, and Li Xiao. MemCalib: Benchmarking and optimizing memory use in LLM agents. *arXiv preprint arXiv:2609.24259*, 2026.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. LightMem: Lightweight and efficient memory-augmented generation. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=dyJ0GWpjJB>.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From RAG to memory: Non-parametric continual learning for large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pp. 21497–21515, 2025.
- Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, and Yafeng Deng. EverMemOS: A self-organizing memory operating system for structured long-horizon reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pp. 45836–45853, 2026a.
- Yulin Hu, Zimo Long, Jiahe Guo, Xingyu Sui, Xing Fu, Weixiang Zhao, Yanyan Zhao, and Bing Qin. OP-Bench: Benchmarking over-personalization for memory-augmented personalized conversational agents. *arXiv preprint arXiv:2601.13722*, 2026b.
- Shuo Ji, Yibo Li, and Bryan Hooi. Memory is reconstructed, not retrieved: Graph memory for LLM agents. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=xRVWftS3ES>.
- Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory OS of AI agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 25961–25970, 2025.

- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023.
- Chris Latimer, Nicoló Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight is 20/20: Building agent memory that retains, recalls, and reflects. *arXiv preprint arXiv:2512.12818*, 2025.
- Kai Li, Xuanqing Yu, Ziyi Ni, Yi Zeng, Yao Xu, Zheqing Zhang, Xin Li, Jitao Sang, Xiaogang Duan, Xuele Wang, Chengbao Liu, and Jie Tan. TiMem: Temporal-hierarchical memory consolidation for long-horizon conversational agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 21700–21720, 2026.
- Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. SimpleMem: Efficient lifelong memory for LLM agents. In *Forty-third International Conference on Machine Learning*, 2026a. URL <https://openreview.net/forum?id=oBgLv5YC6>.
- Jiaqi Liu, Xinyu Ye, Peng Xia, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. EvolveMem: Self-evolving memory architecture via AutoResearch for LLM agents. *arXiv preprint arXiv:2605.13941*, 2026b.
- Qingshan Liu, Guoqing Wang, Wen Wu, Jingqi Huang, Xinqi Tao, Dejia Song, Jie Zhou, and Liang He. MemPro: Agentic memory systems as evolvable programs. *arXiv preprint arXiv:2606.00619*, 2026c.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 13851–13870, 2024.
- NVIDIA. NVIDIA A100 Tensor Core GPU. <https://www.nvidia.com/en-us/data-center/a100/>, 2020. Accessed: 2025-04-01.
- OpenAI. Introducing GPT-5.5. <https://openai.com/index/introducing-gpt-5-5/>, 2026. Accessed: 2026-08-30.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Wenbo Pan, Shujie Liu, Xiangyang Zhou, Shiwei Zhang, Wanlu Shi, Mirror Xu, and Xiaohua Jia. M*: Every task deserves its own memory harness. *arXiv preprint arXiv:2604.11811*, 2026.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–22, 2023.
- Sidharth Pulipaka, Oliver Chen, Manas Sharma, Taaha Saleem Bajwa, Vyas Raina, and Ivaxi Sheth. PersistBench: When should long-term memories be forgotten by LLMs? In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=Z7RhZk13NT>.
- Qwen. Qwen3-Embedding-8B. <https://huggingface.co/Qwen/Qwen3-Embedding-8B>, 2025. Accessed: 2026-05-01.
- Qwen. Qwen3.8-27B. <https://huggingface.co/Qwen/Qwen3.8-27B>, 2026. Accessed: 2026-08-30.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

- Rana Salama, Jason Cai, Michelle Yuan, Anna Currey, Monica Sunkara, Yi Zhang, and Yassine Benajiba. MemInsight: Autonomous memory augmentation for LLM agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 33136–33152, 2025.
- Mohammad Tavakoli, Alireza Salemi, Carrie Ye, Mohamed Abdalla, Hamed Zamani, and J. Ross Mitchell. Beyond a million tokens: Benchmarking and enhancing long-term memory in LLMs. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=y59hf5lrMn>.
- Piaohong Wang, Motong Tian, Jiaxian Li, Yuan Liang, Yuqing Wang, Qianben Chen, Tiannan Wang, Zhicong Lu, Jiawei Ma, Yuchen Eleanor Jiang, and Wangchunshu Zhou. O-Mem: Omni memory system for personalized, long horizon, self-evolving agents. *arXiv preprint arXiv:2511.13593*, 2025.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=pZiyCaVuti>.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, volume 38, pp. 17577–17604, 2025.
- B. Y. Yan, Chaofan Li, Hongjin Qian, Shuqi Lu, and Zheng Liu. General agentic memory via deep research. *arXiv preprint arXiv:2511.18423*, 2025.
- Juexiang Ye, Xue Li, Xinyu Yang, Chengkai Huang, Lanshun Nie, Lina Yao, and Dechen Zhan. MemWeaver: Weaving hybrid memories for traceable long-horizon agentic reasoning. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 12928–12956, 2026.
- Guibin Zhang, Haotian Ren, Chong Zhan, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. MemEvolve: Meta-evolution of agent memory systems. In *Forty-third International Conference on Machine Learning*, 2026a. URL <https://openreview.net/forum?id=qpkG0eKx4v>.
- Weizhi Zhang, Xinyang Zhang, Chenwei Zhang, Liangwei Yang, Jingbo Shang, Zhepei Wei, Henry Peng Zou, Zijie Huang, Zhengyang Wang, Yifan Gao, Xiaoman Pan, Lian Xiong, Jingguo Liu, Philip S. Yu, and Xian Li. PersonaAgent: Bridging memory and action for personalized LLM agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 26421–26439, 2026b.
- Wanjuan Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19724–19731, 2024.
- Yue Zhou, Xiaobo Guo, Belhassen Bayar, and Srinivasan H. Sengamedu. Amory: Building coherent narrative-driven agent memory through agentic reasoning. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics*, pp. 3926–3938, 2026.

A IMPLEMENTATION DETAILS

A.1 CAPABILITY SELECTION AND OFFLINE ANNOTATION

Selecting Capabilities for Our Study. Capability-driven evolution needs a vocabulary that separates recurring memory requirements and makes their progress observable. As an initial step toward capability-driven evolution, we derive five representative capabilities in Table 1 from a broad review of agent memory benchmarks and memory systems (Maharana et al., 2024; Wu et al., 2025; Tavakoli et al., 2026; Zhong et al., 2024; Rasmussen et al., 2025; Chhikara et al., 2025; Xu et al., 2025). Explicit fact recovery, changing states, persistent user constraints, evidence distributed across sessions, and insufficient or conflicting support recur across these benchmarks and systems. These requirements provide a concrete instantiation of the vocabulary used in our study, covering *factual retrieval*, *temporal tracking*, *preference extraction*, *multi-session synthesis*, and *adversarial*. Table 5 connects this vocabulary to existing benchmark question types.

Table 5: Correspondence between benchmark question types and the memory capabilities used in our study. The entries describe primary requirements; individual questions may involve additional capabilities.

Question type	Primary memory capabilities
LoCoMo (Maharana et al., 2024)	
Single-hop; Open-domain knowledge ¹	Factual retrieval
Multi-hop	Multi-session synthesis
Temporal reasoning	Temporal tracking
Adversarial	Adversarial
LongMemEval (Wu et al., 2025)	
Single-session user; single-session assistant	Factual retrieval
Single-session preference	Preference extraction
Multi-session	Multi-session synthesis
Temporal reasoning; knowledge update	Temporal tracking
Abstention variants ²	Adversarial
BEAM (Tavakoli et al., 2026)	
Information extraction	Factual retrieval
Event ordering; temporal reasoning; knowledge update	Temporal tracking
Preference following; instruction following	Preference extraction
Multi-session reasoning; summarization	Multi-session synthesis
Contradiction resolution; Abstention	Adversarial

¹LoCoMo’s open-domain questions require retrieving information about the speaker and combining it with external knowledge. We therefore map this question type to factual retrieval.

²The question identifier ending in `_abs`.

An individual task can require several capabilities. For example, in BEAM (Tavakoli et al., 2026), resolving conflicting statements about whether a user has implemented a project feature requires factual retrieval to recover both claims and adversarial handling to recognize their inconsistency and request clarification. In LongMemEval (Wu et al., 2025), counting the distinct museums or galleries visited in a given month requires multi-session synthesis to combine reported visits and temporal tracking to exclude visits outside that month. These examples motivate the primary–secondary annotation below.

Offline Capability Annotation. Question text alone is insufficient for reliably identifying a task’s memory requirements. For example, a BEAM question asks about a user’s education and specialization in psychology, yet neither detail is provided in the history. The task therefore requires adversarial abstention despite its factual wording. We tested question-only annotation with GPT-5.5 on 700 BEAM-1M questions. The predicted primary capability matched the benchmark-type mapping in Table 5 for only 337 questions (48.14%), indicating the limited reliability of question-only annotation in this setting.

We therefore provide the annotator with the question, reference answer and evidence. The answer clarifies the expected response, while the evidence identifies the information needed to support it. We use LLM-based annotation so that the procedure also applies when benchmark-specific question types are unavailable. Given these inputs and the capability definitions, we use GPT-5.5 to assign one primary capability and up to two distinct secondary capabilities using the prompt template below. The primary label captures the requirement whose failure would most directly prevent a correct answer; secondary labels capture additional requirements. We reuse valid annotations across runs.

Capability labels are used only to aggregate capability-level feedback and select the next capability to evolve; they are never passed to the memory program. We therefore annotate only the training and validation sets, with no annotation needed for the test set. At test time, the evolved program receives the history and question, preserving the same task information available to the baselines for a fair comparison. Native benchmark types will be used to group the test set for capability-level reporting, as detailed in Section B.1.

Prompt for offline capability annotation

System.

You are a classifier agent. Classify the question according to the memory-related capabilities it requires. Return exactly the requested JSON object.

Task.

Determine which long-term-memory capabilities are required to answer the current question correctly. Use the question, golden evidence, and golden answer together to identify the memory operation the question tests.

Capabilities.

- `factual_retrieval`: locate and preserve an explicit fact, name, exact value, list item, or other localized detail. Use it as primary when direct information recovery is the central requirement.
- `temporal`: order events, compare times, determine recency, or track an old state, a new state, and a knowledge update.
- `preference_extraction`: recover and apply a user’s lasting preference, prior instruction, constraint, or standing need.
- `multi_session_synthesis`: connect, aggregate, or summarize evidence distributed across multiple sessions or events.
- `adversarial`: abstain when memory lacks supporting evidence, or cautiously identify and resolve contradictory or misleading evidence.

Choose exactly one primary capability: the capability whose failure would most directly prevent a correct answer. Add zero, one, or two distinct secondary capabilities only when they are also genuinely required. For example, a contradiction whose resolution depends on recency can use adversarial as primary and temporal as secondary; a changed preference can use preference extraction and temporal.

Question:

[question]

Golden evidence:

[reference snippets serialized as JSON]

Golden answer:

[reference answer]

Output.

Return exactly one JSON object and no other text. Allowed labels are the five identifiers above; secondary labels must be distinct and different from the primary label.

```
{"primary": "<one allowed capability>",
 "secondary": ["<zero to two other capabilities>"]}
```

A.2 EVIDENCE-BASED DIAGNOSTIC METRICS

During the cold-start stage, we diagnose the memory pipeline by tracing each reference evidence snippet from its source dialogue segment through extraction, indexing, and retrieval. Specifically, we examine whether the relevant information is preserved in the extracted memory units, effectively represented and accessible through the index, and ultimately retrieved for the actual question. These diagnostics allow us to distinguish weaknesses in *Extraction*, *Indexing*, *Planning* and *Retrieval* across capabilities.

The three diagnostics use the same embedding-based matching function. For each task x , let E_x denote the set of distinct reference evidence snippets annotated by the dataset. Each $e \in E_x$ can be uniquely mapped to its source dialogue segment after text normalization, from which we can obtain the corresponding extracted memory units U_e . To measure how well a set of text items preserves or exposes a reference evidence snippet, we compare the evidence with the most similar item in the set. Specifically, for an L2-normalized embedding function ϕ and a set of text items Z , we define the maximum matching similarity as:

$$\text{sim}_{\max}(e, Z) = \begin{cases} \max_{z \in Z} \phi(e)^\top \phi(z), & Z \neq \emptyset, \\ 0, & Z = \emptyset. \end{cases} \quad (8)$$

This score measures whether the information in evidence e is represented by at least one item in Z . Let R_e denote the text blocks retrieved from the memory index when the original evidence e is used as an oracle query, following the standard query planning and retrieval procedure. Let R_x denote the text blocks retrieved by the memory program for the actual question x . We then compute:

$$d_{\text{ext}}(x) = \frac{1}{|E_x|} \sum_{e \in E_x} \text{sim}_{\max}(e, U_e), \quad (9)$$

$$d_{\text{idx}}(x) = \frac{1}{|E_x|} \sum_{e \in E_x} \text{sim}_{\max}(e, R_e), \quad (10)$$

$$d_{\text{ret}}(x) = \frac{1}{|E_x|} \sum_{e \in E_x} \text{sim}_{\max}(e, R_x). \quad (11)$$

Here, d_{ext} measures whether the relevant evidence is preserved after extraction, d_{idx} measures whether the preserved information can be effectively accessed through the index using an oracle evidence query, and d_{ret} measures whether the memory program retrieves the relevant evidence for the actual question. We report these diagnostics separately for each capability. Comparing them helps localize weaknesses to memory extraction, indexing, or query planning and retrieval. For example, a high d_{ext} but substantially lower d_{idx} suggests that the evidence is preserved in memory units but not effectively indexed or accessed. A high d_{idx} but low d_{ret} instead points to problems in query formulation, query planning, or retrieval.

Two length statistics provide additional context: the average number of characters of the memory units and retrieved content. The former helps assess whether extraction produces overly verbose memory units, while the latter characterizes the amount of information exposed to the answer model and provides context for interpreting retrieval quality.

A.3 DIAGNOSTIC CASE PRESENTATION AND COMPRESSION

Diagnostic cases provide the context needed to understand and diagnose errors in the memory pipeline, especially in later evolution iterations. To support detailed analysis, each case presents a comprehensive view of the information relevant to the task. Each case includes the primary and secondary capability labels, question, reference answer, predicted answer, task score, query plan, retrieved text, and a reference evidence view. The evidence view pairs each reference evidence snippet with its original source text and the memory units in which it is represented. This pairing helps distinguish missing evidence from evidence that was preserved in memory but poorly indexed, retrieved, or interpreted. Diagnosis and coding receive the same compact case material, allowing each proposed change to be traced back to the evidence that motivates it.

Full source segments and retrieved passages can be lengthy, making diagnostic cases unnecessarily large. We therefore compress the material presented to the evolution model while preserving the

evidence needed for diagnosis. We use a budget of 2400 characters per displayed excerpt and retain shorter texts in full. For longer texts, we divide them into source records, paragraphs, or sentences, while preserving session, date, and role headers and keeping fenced code blocks intact. A greedy selection procedure then selects spans that maximize coverage of the question and reference answer, corresponding reference evidence, and the model prediction. Span relevance combines semantic cosine similarity and stemmed non-stopword Dice overlap, weighted at 0.75 and 0.25, respectively. The question–answer cue has weight 1, the evidence snippets have a total weight of 1, and the prediction has weight 0.5. Including the prediction helps retain evidence that explains an incorrect answer, such as conflicting numeric values.

Selected spans are restored to their original source order and separated by explicit omission markers (...). For extracted memory, we always retain at least the best-matching unit for each reference evidence snippet, add other relevant units when space permits, and deduplicate repeated displays. These choices preserve the connections among reference evidence, its memory representation, and the model prediction while removing repeated or unrelated context. We evaluate the effect of context reduction on PRISMEM’s real traces from BEAM and find that the average case-context size decreases from 16,776 to 9,428 tokens, corresponding to a 43.8% reduction.

A.4 VALIDATE GENERATED PROGRAMS

Generated memory program revisions can fail before they can be evaluated. We check program completeness, interface compatibility, and basic execution behavior before allocating task model calls, so readily detectable errors can be corrected early. The checks proceed from inexpensive source inspection to bounded synthetic execution.

1. **Source and module completeness.** The patched source must be nonempty, parse as Python, use allowed imports, and avoid evaluation-side interfaces. The module must load successfully and define `MemoryUnit`, `Query`, `Memory`, and the three nonempty extraction, planning, and answer instruction strings. A patch must make an actual source change.
2. **Schema and signature compatibility.** `MemoryUnit` and `Query` must be nonempty data-classes with supported JSON-compatible fields: strings, integers, floats, booleans, string lists, and their optional forms. The constructor, `index`, and `retrieve` must preserve the required synchronous signatures. Structured outputs are checked for missing or extra fields, invalid types, and nonfinite numeric values.
3. **Functional smoke tests.** A local stub backend exercises construction, indexing a session with synthetic units, indexing an empty-unit session, and retrieval with a valid query. Indexing must return `None`; retrieval must return a string within the 16K-character context limit. These tests require no task-LLM inference.
4. **Synthetic complexity checks.** A larger probe indexes ~ 2 K synthetic units across 100 batches and executes two queries with 4096-dimensional embeddings. It checks local computation and backend-call counts. A local CPU-work budget and an outer process timeout bound stalled or unexpectedly expensive execution.

These checks catch malformed schemas, incomplete patches, interface mismatches, and basic execution failures before full task evaluation.

A.5 MEMORY INTERFACE.

The memory interface defines the scope of modifications during self-evolution. We decompose the memory program into five components: *Extraction*, *Indexing*, *Planning*, *Retrieval*, and *Answer*, enabling precise problem localization and targeted modifications by the model.

The five components are defined as follows: *Extraction* converts history segments into structured memory units $U = P.\text{extract}(\mathcal{H})$; *Indexing* organizes these units U into persistent memory $M = P.\text{index}(U, \mathcal{H})$; *Planning* translates the question q into a structured query unit $Q = P.\text{plan}(q)$; *Retrieval* executes the query Q against persistent memory M to return relevant content $R = P.\text{retrieve}(M, Q)$; and *Answer* instructs the task model to use the retrieved content R and question q to generate a high-quality answer $\hat{y} = P.\text{answer}(q, R)$. Evolution can modify the

schemas of memory and query units, as well as the code and instructions for these five memory components.

Here are the memory interface definitions provided in the prompt.

Memory interface definition

You are designing a Memory Program that implements three classes:

1. MemoryUnit (dataclass): Defines what information is captured as memory units from source text.

- Must be a `@dataclass` with typed fields
- An external LLM will populate several instances by generating JSON matching your field definitions
- **Field types MUST be JSON-compatible:** use only `str`, `int`, `float`, `bool`, `list[str]`, and their Optional forms
- Do NOT use `datetime`, `tuple`, `bytes`, `dict`, nested `dataclasses`, or custom objects — the fixed harness cannot construct them from the generated JSON
- Use `field(metadata={"description": "..."})` to describe fields — descriptions are shown to the LLM that populates instances

2. Query (dataclass): Defines what parameters are used when retrieving from memory.

- Must be a `@dataclass` with typed fields
- An external LLM will populate one instance by generating JSON matching your field definitions
- Same JSON-compatible type restriction and field description support as `MemoryUnit`

3. Memory (class): The core memory system.

```
__init__(self, backend)
```

Receives a Backend with:

```
backend.ask(prompt, system_prompt=None,  
            json_object=False) -> str
```

LLM for reasoning, summarization, or reranking

```
backend.embed(texts: list[str]) -> np.ndarray
```

One L2-normalized embedding row per input text

```
index(self, units: list[MemoryUnit], raw_text: str) -> None
```

Incrementally index one source session. `units` contains all valid `MemoryUnits` generated from that session and may be empty. `raw_text` is the raw session text with a timestamp. Calls occur in chronological order. In this function, you may merge true duplicates, normalize references to the same entity, preserve timestamps and the order of distinct, updated, or conflicting facts, and build dense, lexical, exact-value, graph, metadata, or combined indexes for retrieval.

```
retrieve(self, query: Query) -> str
```

Retrieve relevant information as a string. Use the `Query` fields to search the relevant dense, lexical, exact-value, graph, or metadata indexes, rank and combine matches, remove repeated results, and return the useful context within `OUT_CHAR_BUDGET`.

Allowed imports: `json`, `re`, `math`, `hashlib`, `collections`, `dataclasses`, `typing`, `datetime`, `textwrap`, `numpy`. The execution environment provides `OUT_CHAR_BUDGET`, the enforced retrieval-character limit.

Runtime constraints

These limits are enforced during evaluation. Violating them makes the evaluation fail.

- **retrieve() output limit:** `Memory.retrieve()` must return at most **16000 characters**.

- `retrieve()` is called once per question, and must be read-only and independent of prior query calls.

Instruction constants (required)

Three module-level string constants provide the natural-language instructions used in task-LLM prompts:

- `MemoryExtractionInstruction`: Instruction for `MemoryUnit` generation. Tells the task LLM to extract atomic facts, procedures, relationships, decisions, and preferences from each segment, resolve references, and preserve names, exact values, dates/times, and lists. The `MemoryUnit` schema is provided separately — no need to describe field names or types here.
- `QueryPlanningInstruction`: Instruction for Query generation. Tells the task LLM how to formulate retrieval queries and tailor them to the capabilities required by the question, including splitting multi-aspect questions when useful. The Query schema is provided separately — no need to describe field names or types here.
- `AnswerInstruction`: Instruction for answer generation. Controls answer format, length, and style, and guides how retrieved content is interpreted and used. Apply question-specific strategies when needed, including distinguishing user statements from assistant suggestions, resolving updates or contradictions using source, time, and scope, counting distinct events rather than memory units or repeated mentions, and synthesizing facts across sessions. Preserve relevant chronology and specific changes for progression questions. Ensure that information needed for source, time, or event identification is preserved in memory and exposed during retrieval.

`MemoryExtractionInstruction`, `QueryPlanningInstruction`, and `AnswerInstruction` must not be empty.

A.6 REVISION-AWARE EXECUTION REUSE

A local revision often leaves most of the memory pipeline unchanged. We compare the generated candidate’s components with those of its parent and begin re-execution at the earliest affected stage, as shown in Table 6. For example, a new ranking rule can reuse extracted units and the built memory, whereas a changed extraction instruction requires new units and all downstream computations. This dependency-aware reuse avoids unnecessary LLM calls while preserving evaluation coverage.

Table 6: Re-execution boundaries for a candidate relative to its parent. For changes spanning several components, the earliest affected stage determines what can be reused.

Changed component	Re-execution begins at	Reusable work
Memory-unit schema or extraction stage	Extraction	Segmentation
Constructor, index, or helpers	Indexing	Segmentation and extracted units
Query schema, planning, or retrieval	Query planning	Segmentation, extraction, and indexed memory
Answer instruction only	Answer generation	Built memory, query plan, and retrieved context

Reusable state is copied before being attached to the candidate, keeping parent and child memory states separate. Changes to shared helpers invalidate indexing conservatively because those helpers may affect memory construction. Reuse follows the program’s execution dependencies and the actual inputs to cached computations.

A.7 SEED MEMORY PROGRAM

The initial seed memory program is deliberately simple, providing a broad search space. It appends extracted units in input order and returns their concatenation, truncated to the context budget, with-

out query-dependent ranking. The execution environment fixes the output character budget at 16K (OUT_CHAR_BUDGET = 16000) across evolution runs. The implementation is shown below.

Initial seed memory program used for PRISMEM

```
from dataclasses import dataclass, field

MemoryExtractionInstruction = (
    "Extract the key information from the text "
    "into several memory units."
)
QueryPlanningInstruction = (
    "Given the following question, generate a query "
    "to retrieve relevant memory."
)
AnswerInstruction = (
    "Based on the retrieved memory and the original question, "
    "provide a short answer without explanation."
)

@dataclass
class MemoryUnit:
    memory_text: str = field(
        metadata={"description": "One piece of information
        worth remembering from the text"}
    )

@dataclass
class Query:
    query_text: str = field(
        metadata={"description": "A query for retrieving
        information relevant to the question"}
    )

class Memory:
    def __init__(self, backend):
        self.backend = backend
        self.memories: list[str] = []

    def index(self, units: list[MemoryUnit], raw_text: str) ->
    None:
        self.memories.extend(unit.memory_text for unit in units
    )

    def retrieve(self, query: Query) -> str:
        return "\n\n".join(self.memories)[:OUT_CHAR_BUDGET]
```

A.8 PSEUDOCODE OF PRISMEM

We provide the pseudocode of PRISMEM below, illustrating its execution flow.

Algorithm 1 PRISMEM: Capability-Driven Memory Evolution

Require: Seed P_0 ; datasets $\mathcal{S}_{\text{train}}, \mathcal{S}_{\text{valid}}$; capability set $\mathcal{C}$; iterations for three stages T_1, T_2, T_3 with $T = T_1 + T_2 + T_3$; case budget K ; thresholds δ, θ .

Ensure: Final memory program G_T .

```
1:  $\mathcal{W} \leftarrow \{P_0\}$ ; EVALUATE( $P_0, \mathcal{S}_{\text{train}}, \mathcal{S}_{\text{valid}}$ )
2: for  $t = 1, \dots, T_1$  do ▷ Stage I: Cold Start
3:    $P \leftarrow \text{SELECT}(\mathcal{W}, t)$ ;  $M \leftarrow \text{COMPUTEMETRICS}(P, \mathcal{S}_{\text{train}})$ 
4:    $d \leftarrow \text{DIAGNOSE}(P, M)$ ;  $P' \leftarrow \text{REVISE}(P, d, M)$ 
5:   if  $P' \neq \perp$  and EVALUATE( $P', \mathcal{S}_{\text{train}}, \mathcal{S}_{\text{valid}}$ ) succeeds then
6:      $\mathcal{W} \leftarrow \mathcal{W} \cup \{P'\}$ 
7:   end if
8: end for
9:  $P_{\text{base}} \leftarrow \arg \max_{P \in \mathcal{W}} s(P; \mathcal{S}_{\text{valid}})$ ;  $\mathcal{P}_{T_1} \leftarrow \{P_{\text{base}}\}$ ;  $G_{T_1} \leftarrow P_{\text{base}}$ 
10:  $P_{T_1}^*(c) \leftarrow P_{\text{base}}$  for every  $c \in \mathcal{C}$ 
11: for  $t = T_1 + 1, \dots, T_1 + T_2$  do ▷ Stage II: Capability Refinement
12:    $c_t \leftarrow \text{SELECTCAPABILITY}(\{P_{t-1}^*(c)\}_{c \in \mathcal{C}}, t)$  ▷ Priority  $\rho_t(c)$  via Equation (6)
13:    $P_t \leftarrow P_{t-1}^*(c_t)$ 
14:    $\mathcal{B}_t \leftarrow \text{TOPK}(\{x_i \in \mathcal{S}_{\text{train}}^{c_t} : s(P_t; x_i) < \delta\}, w_t, \delta, K)$  ▷ Curate via Equation (7)
15:    $\mathcal{Z}_t \leftarrow \{\text{CASE}(P_t, x) \mid x \in \mathcal{B}_t\}$ 
16:    $d_t \leftarrow \text{DIAGNOSE}(P_t, c_t, \mathcal{Z}_t)$ 
17:    $P' \leftarrow \text{REVISE}(P_t, d_t, \mathcal{Z}_t)$ 
18:   if  $P' \neq \perp$  and  $s(P'; \mathcal{S}_{\text{train}}^{c_t}) > s(P_t; \mathcal{S}_{\text{train}}^{c_t})$  then
19:     if EVALUATE( $P', \mathcal{S}_{\text{train}}, \mathcal{S}_{\text{valid}}$ ) succeeds then ▷ Full-suite evaluation
20:        $\mathcal{P}_t \leftarrow \mathcal{P}_{t-1} \cup \{P'\}$ 
21:     else
22:        $\mathcal{P}_t \leftarrow \mathcal{P}_{t-1}$ 
23:     end if
24:   else
25:      $\mathcal{P}_t \leftarrow \mathcal{P}_{t-1}$ 
26:   end if
27:    $P_t^*(c) \leftarrow \arg \max_{P \in \mathcal{P}_t} s(P; \mathcal{S}_{\text{valid}}^c)$  for every  $c \in \mathcal{C}$ 
28:    $G_t \leftarrow \arg \max_{P \in \mathcal{P}_t} s(P)$ 
29: end for
30:  $\mathcal{E} \leftarrow \emptyset$ ;  $\mathcal{B} \leftarrow \emptyset$ ;  $T' \leftarrow T_1 + T_2$  ▷ Stage III: Integration
31:  $G_{T'} \leftarrow G_{T'}$ ;  $\mathcal{T} \leftarrow \text{EVOLUTIONTRACE}(\mathcal{P}_{T'})$ ;
32: for all  $c \in \mathcal{C}$  where  $s(P_{T'}^*(c); \mathcal{S}_{\text{train}}^c) > s(G_{T'}; \mathcal{S}_{\text{train}}^c)$  do
33:    $\hat{x}_c \leftarrow \arg \max_{x \in \mathcal{S}_{\text{train}}^c} [s(P_{T'}^*(c); x) - s(G_{T'}; x)]$ 
34:    $\mathcal{E} \leftarrow \mathcal{E} \cup \{\text{CASE}(P_{T'}^*(c), \hat{x}_c), \text{CASE}(G_{T'}, \hat{x}_c)\}$ 
35:    $\mathcal{B} \leftarrow \mathcal{B} \cup \{\hat{x}_c\}$ 
36: end for
37: if  $\mathcal{E} \neq \emptyset$  then
38:    $\Pi \leftarrow \text{PLANINTEGRATION}(G_{T'}, \{P_{T'}^*(c)\}_{c \in \mathcal{C}}, \mathcal{T}, \mathcal{E})$  ▷ Generate plan
39:   for  $t = T' + 1, \dots, T$  do
40:      $P_t \leftarrow \text{EXECUTEINTEGRATE}(G_{T'}, \Pi, \mathcal{E})$  ▷ Coding with plan and paired cases
41:     if  $P_t \neq \perp$  and  $s(P_t) > s(G_{T'})$  and  $\max_{c \in \mathcal{C}} [1 - s(P_t; \mathcal{S}_{\text{train}}^c) / s(G_{T'}; \mathcal{S}_{\text{train}}^c)] \leq \theta$  then
42:        $G_T \leftarrow P_t$ 
43:     break
44:   else
45:      $\mathcal{E} \leftarrow \mathcal{E} \cup \{\text{CASE}(P_t, x) \mid x \in \mathcal{B}\}$  ▷ Accumulate cases
46:   end if
47: end for
48: end if
49: return  $G_T$  ▷ Final unified program
```

B EXPERIMENTAL DETAILS

B.1 DATASET SPLITS AND CAPABILITY COMPOSITION

We evolve memory programs on the shorter-history BEAM-100K and LongMemEval-S splits ($\sim 100K$ tokens) and evaluate their frozen implementations on the million-token BEAM-1M and LongMemEval-M splits. This setup reduces the risk of overestimating evolutionary performance due to overfitting and provides a more direct evaluation of the evolution method’s ability to generalize to million-token histories.

For BEAM, the evolution split contains 160 training questions from 8 conversations and 120 validation questions from 6 conversations, while the test set contains all 700 BEAM-1M questions from another 35 conversations. For LongMemEval, each question is associated with a specific conversation. To reduce evaluation cost while maintaining a balanced question-type distribution, we stratify by question type and randomly sample 60 questions for training and 60 for validation in total. Following the same stratified sampling procedure, we construct a 200-question subset of LongMemEval-M with different histories for testing. The frozen program constructs memory from each test history with its implementation held fixed.

Table 7 reports the capability counts and average history length of each split. Training and validation labels are assigned by the LLM annotator in Section A.1. Test questions are grouped using the native question-type mapping in Table 5.

Table 7: Sizes, primary capability composition, and average history length of each dataset split. F, T, P, M, and A denote factual retrieval, temporal tracking, preference extraction, multi-session synthesis, and adversarial, respectively. Average history length is measured with the Qwen3.8-27B tokenizer.

Dataset	Split	F	T	P	M	A	Total	Avg. tokens (K)
BEAM-100K	Train	20	43	32	33	32	160	125.11
BEAM-100K	Validation	18	32	24	22	24	120	146.83
BEAM-1M	Test	70	210	140	140	140	700	1,089.57
LongMemEval-S	Train	19	17	11	9	4	60	109.40
LongMemEval-S	Validation	20	17	8	8	7	60	109.40
LongMemEval-M	Test	64	64	30	32	10	200	1,079.04

B.2 PARAMETER SETTINGS

Model Inference Parameters. For task models, we disable thinking or use low thinking effort with a maximum of 8,192 tokens. For diagnosis and coding models, we enable thinking with medium reasoning effort and a maximum of 65,536 tokens. All methods use the same inference settings. For benchmark judging, we use the same model as the task model and set the temperature to 0 for deterministic outputs. We test Qwen3.8-27B and GPT-5.5 in our experiments.

Table 8: Parameter settings of PRISMEM we used in experiments.

Parameter	Value
Stage II: Capability refinement	
Case score threshold δ	0.8
Refinement case budget K	5
Stage III: Integration	
Capability-wise score degradation ratio θ	0.1
Integration case budget	1 per capability

Algorithm Parameters. We use an evolution budget of $T = 20$ rounds for all evolutionary methods. For PRISMEM, we allocate $T_1 = 5$ rounds to cold start, $T_2 = 12$ to capability refinement, and $T_3 = 3$ to final integration. Table 8 lists the parameter values used for stages II and III.

C ADDITIONAL EXPERIMENTAL RESULTS

C.1 RESULTS ON GPT-5.5

Table 9 shows the performance of different methods on BEAM using GPT-5.5. The results preserve the same trends observed with Qwen3.8-27B (Table 2), demonstrating the robustness of PRISMEM across different underlying models.

Table 9: Performance on BEAM with GPT-5.5.

Method	Overall Score (%) $\uparrow$	Avg. #Tokens (K) $\downarrow$
Mem0	57.77 ± 0.17	180.21 ± 0.12
A-MEM	60.92 ± 0.41	352.11 ± 0.25
HippoRAG2	57.73 ± 0.50	193.52 ± 0.18
SimpleMem	52.36 ± 0.55	121.39 ± 0.69
LightMem	46.68 ± 0.05	94.05 ± 0.19
M*	60.75 ± 2.53	97.47 ± 4.75
EvolveMem	46.77 ± 1.56	94.39 ± 3.08
PRISMEM	67.09 ± 1.79	99.36 ± 3.62

C.2 TRANSFER EXPERIMENTS

We examine whether PRISMEM’s evolved memory programs transfer beyond the model and dataset used during evolution. We evaluate memory programs evolved with Qwen3.8-27B on BEAM-100K. For model transfer, we evaluate these programs with GPT-5.5 on BEAM-1M. For dataset transfer, we retain Qwen3.8-27B and evaluate on LongMemEval-M. The other settings remain the same as those in the main experiments. The results are summarized in Table 10 and Table 11.

Table 10: Model transfer evaluated with GPT-5.5 on BEAM-1M. All programs are evolved on BEAM-100K. Overall scores (%) are reported as “mean $\pm$ std” over three runs.

Method	Evolved with Qwen3.8-27B (transfer)	Evolved with GPT-5.5 (target model)
PRISMEM	67.25 ± 1.35	67.09 ± 1.79

Table 11: Dataset transfer evaluated with Qwen3.8-27B on LongMemEval-M. All programs are evolved with Qwen3.8-27B. Overall scores (%) are reported as “mean $\pm$ std” over three runs.

Method	Evolved on BEAM (transfer)	Evolved on LongMemEval (target dataset)
PRISMEM	76.50 ± 1.67	75.50 ± 1.80

These results show that the evolved memory programs maintain strong performance even when transferred to different models and datasets without additional program optimization, demonstrating the robustness and generalizability of the capability-driven evolution approach.

C.3 CAPABILITY ROUTER EXPERIMENTS

We compare trace-guided integration with routing among capability-best programs from the same PRISMEM evolution trajectory on BEAM. The *question-only router* uses Qwen3.8-27B to predict the primary capability from the question alone, since reference answers and evidence are unavailable at test time. The *oracle-label router* uses the benchmark question types mapped to capabilities in Table 5. Both routers select the corresponding capability-best program from the evolution run. In

contrast, integration uses a single program for all questions without capability labels at test time. Overall scores are computed over all 700 questions in BEAM-1M.

Table 12: Routing versus integration on BEAM-1M with Qwen3.8-27B.

Strategy	Overall Score (%)
Question-only router	57.74 $\pm$ 1.21
Oracle-label router	62.27 $\pm$ 1.14
PRISMEM	63.39$\pm$ 1.02

Table 12 shows that integration outperforms question-only routing by 5.65 pp and oracle-label routing by 1.12 pp. Question-only routing is limited by the difficulty of inferring memory requirements from the question alone during test time, without access to the reference answer and supporting evidence; its predicted labels match the oracle mapping for only 48% of the total 700 questions. Although oracle-label routing uses accurate capability labels, it still selects individual programs optimized primarily along specific capability directions. Since a question may require multiple capabilities, these programs may provide unbalanced coverage of its requirements. In contrast, trace-guided integration combines complementary mechanisms within a shared program and reconciles their conditions of use, resulting in stronger overall performance. These results support the value of integrating useful capability-specific revisions into a shared memory program.

D DETAILED EVOLUTION TRACE

Table 13 summarizes key evolution iterations of PRISMEM evolved on BEAM with Qwen3.8-27B, including the diagnosed failure, resulting revision, and modified memory program components. Blue tags denote the capability targeted by each evolution step, while gray letter boxes mark capabilities for which the resulting node achieves the best score (specialist).

Table 13: BEAM evolution traces with failure diagnoses, revision summaries, and current specialist. **F**: factual retrieval, **T**: temporal tracking, **P**: preference extraction, **M**: multi-session synthesis, **A**: adversarial.

Evolution Trace	Failure diagnosis	Revision summary	Current best for
Stage I: Cold-start revisions			
n05	A single flat query could not capture multiple semantic aspects of synthesis questions, causing retrieval to over-concentrate on one topic while missing complementary evidence scattered across sessions.	Decompose multi-aspect questions into focused sub-queries and retrieve each aspect independently with proportional budget allocation and cross-query deduplication. schema planning retrieval answering	M A F T P
Stage II: Capability-directed refinement			
n05→n06 Synthesis	Over-atomized memory units fragmented each logical item into multiple small facts, causing related evidence to appear as an unstructured list and hindering distinct-item counting and cross-session progression synthesis.	Add a topic label to each memory unit and group retrieved evidence by topic, so related facts are presented as coherent blocks for counting and multi-session synthesis. schema extraction indexing retrieval answering	F

Evolution Trace	Failure diagnosis	Revision summary	Current best for
n05→n07 Temporal	Session-level temporal provenance was lost during indexing, so retrieved memories lacked discussion-time context and the model could not reliably distinguish statement order from event dates.	Add session date to each memory unit, recover it from the source session during indexing, and expose explicit date markers in retrieved evidence while preserving relevance-based ranking. schema extraction indexing retrieval answering helpers	F T P
n05→n08 Adversarial	Memory units lacked speaker provenance, causing user-confirmed facts to be conflated with assistant recommendations or hypotheticals and obscuring genuine contradictions between user statements.	Add explicit speaker labels to memory units and propagate them through retrieval, then constrain answering to treat user-sourced units as direct evidence and surface conflicting user statements without resolution. schema extraction indexing retrieval answering	A
n05→n09 Synthesis	Per-unit relevance ranking favored locally similar facts while dropping weaker but essential progression evidence, and the remaining units were returned as a flat list, preventing reconstruction of coherent cross-session threads.	Add topic-level thread labels, group memories by topic and session order, retrieve matched threads as chronological blocks, and use relevance ranking only to fill remaining uncovered evidence. schema extraction indexing retrieval answering helpers	P
n09→n10 Preference	Standing preferences were retrieved only through topical relevance, so cross-cutting user constraints were often omitted when the current query concerned a different domain.	Add an explicit standing-preference flag and reserve retrieval capacity for these units, ensuring persistent user constraints are always exposed alongside task-specific evidence. schema extraction indexing retrieval answering helpers	M
n07→n13 Synthesis	The AnswerInstruction lacked explicit synthesis rules for organizing retrieved evidence across topics and time, leading to fragmented narratives and inconsistent handling of counts, cumulative values, and distinct focus areas.	Add concise answer-side guidance to build chronological topic narratives, distinguish duplicates from cumulative or additive values, and enumerate distinct areas consistently. answering	F
n07→n15 Preference	Retrieved user preferences were treated as passive context rather than binding response constraints, causing the answer LLM to ignore required formatting and presentation behaviors.	Strengthen the AnswerInstruction to enforce retrieved standing preferences as mandatory constraints on response structure and content. answering	P

Evolution Trace	Failure diagnosis	Revision summary	Current best for
n07→n17 Adversarial	Existence questions were planned primarily around affirmative evidence, causing explicit negations to receive low retrieval priority and making user-level contradictions easy to miss.	Make existence-query planning contrastive by retrieving both affirmative and negative evidence, and require the answer LLM to surface both sides when they conflict. planning answering	A T
Stage III: Final integration			
n20	Capability specialists captured complementary strengths, but their mechanisms were not directly composable: aggressive topic grouping, preference enforcement, and aggregation rules could degrade relevance, temporal ordering, or contradiction handling when combined.	Merge the complementary mechanisms conservatively by adding topic and standing-preference metadata, lightweight topic-aware scoring, and concise synthesis/preference rules while preserving flat relevance retrieval and adversarial safeguards. schema extraction indexing retrieval answering helpers	

E CASES AND SUGGESTED REVISIONS ON DIFFERENT CAPABILITIES

We examine three cases from memory evolution trajectories on BEAM, each contrasting two revisions with different capability effects and a third revision that addresses both needs. These cases show that different capabilities may suggest different revisions, yet they are not inherently incompatible: by comparing the strengths and limitations of revisions across capabilities, integration can identify and combine complementary improvements. PRISMEM leverages evolved programs and their evaluation results across capabilities to guide this integration and improve overall performance.

E.1 CASE 1: STORE EACH INTERACTION AS A WHOLE VERSUS EACH FACT SEPARATELY.

Temporal tracking questions compare events or values at different points in a user’s history, requiring the memory to preserve relationships among related details. The diagnosis found that a single request or calculation was split across multiple memory units, causing its steps, values, and outcome to be treated as separate interactions. The model then revised the extraction instruction to keep each complete interaction in one unit. This improved temporal tracking by preserving relationships across events. However, grouping related information also combined target statements with long, less informative context, causing the answer LLM to overlook contradictions with opposing statements elsewhere in the history. Temporal tracking increased by 3.41 pp, while adversarial handling decreased by 14.06 pp.

Adversarial questions require identifying conflicting claims, such as a user stating both that they have practiced a topic and that they have never practiced it. The diagnosis found that distinct factual statements were sometimes omitted during extraction, motivating a revision to the extraction instruction that preserves each distinct fact independently and recovers missing denials in adversarial questions. However, separating facts also dispersed related values across memory units, making it harder to associate them with the events that produced them. Adversarial handling increased by 12.89 pp, while temporal tracking decreased by 6.90 pp.

These cases show that temporal tracking benefits from preserving relationships among events and values, while contradiction detection requires opposing claims to remain independently accessible. One can refine the extraction instruction to ask the model to *preserve a complete plan, process, or timeline within one memory unit while storing contradictory claims separately*. For example, a problem count and its associated score remain together, while a denial of prior practice remains

separate from evidence of completed practice. This revision improved both capabilities: temporal tracking increased by 0.87 pp, and adversarial handling increased by 2.73 pp.

E.2 CASE 2: RECONSTRUCT CHRONOLOGICAL ORDER VERSUS REQUIRE EXPLICIT CAUSAL LINKS.

Temporal tracking questions require reconstructing the order of events across conversations. The diagnosis found that answers grouped concrete events into broad activity categories, obscuring their sequence. The model revised the answer instruction to organize specific events in chronological session order, making the sequence clearer. However, this sometimes led the model to infer a causal relationship from temporal order, treating an event that occurred earlier as having caused a later event. This caused adversarial performance to decrease by 9.38 pp, while temporal tracking increased by 5.61 pp.

Adversarial questions about causal influence require checking whether the history explicitly connects a stated cause to its effect. The diagnosis found that chronological ordering of related events was sometimes treated as evidence of causal influence. The model revised the answer instruction to require an explicit causal link and abstain when such evidence was absent. However, this overly strict criterion caused some temporal questions requiring an explicitly stated sequence to be incorrectly rejected. Adversarial handling increased by 8.20 pp, while temporal tracking decreased by 4.25 pp.

These cases highlight two distinct requirements for relating events: temporal tracking can reconstruct order from dated records, whereas causal influence requires explicit evidence connecting cause and effect. A good answer instruction should ask the model to *organize answers to temporal and progress questions around the user’s specific events in chronological session order, while checking for an explicit connection between cause and effect in causal and influence questions*. This revision improved both capabilities: temporal tracking increased by 2.71 pp, and adversarial handling increased by 8.59 pp.

E.3 CASE 3: INCREASE RETRIEVAL DIVERSITY VERSUS TIGHTEN RELEVANCE FILTERING.

Multi-session synthesis questions require covering different relevant aspects of a user’s history. The diagnosis found that retrieval repeatedly selected similar records from one dominant topic, leaving other aspects underrepresented. The model therefore revised retrieval to use maximal marginal relevance (MMR), balancing relevance to the question with diversity among selected records. This broadened the evidence available for synthesis, but could also introduce records from other events, making it harder to associate dates and updates with the target event in temporal questions. Thus, multi-session synthesis increased by 7.08 pp, while temporal tracking decreased by 5.95 pp.

Temporal tracking questions instead require closely related records about the target event to compare dates and track updates. The diagnosis found that records about nearby events interfered with identifying the correct timeline. The model raised the candidate relevance threshold for MMR (e.g., $0.6 \rightarrow 0.7$), focusing retrieval on records more closely related to the question. However, stricter filtering also removed records covering other relevant aspects, narrowing the evidence available for broad synthesis. Temporal tracking therefore increased by 5.87 pp, while multi-session synthesis decreased by 5.62 pp.

These cases highlight two retrieval requirements: broad coverage for multi-session synthesis and focused retrieval of related records for temporal tracking. A good retrieval revision should be able to *adjust the MMR balance according to the predicted question type: synthesis questions favor greater diversity (e.g., 0.6 MMR weight), while temporal and progression questions favor relevance to retain closely related records (e.g., 0.7 MMR weight)*. This revision improved both capabilities: multi-session synthesis increased by 3.92 pp, and temporal tracking increased by 3.70 pp.

F PROMPT CATALOG

We present the reflection, refinement coder, planning, and integration coder prompts used in PRISMEM. The reflection and refinement coder templates are used in Stage I (cold start) and Stage

II (capability refinement), while the planning and integration coder templates are used in Stage III (integration).

Reflection prompt template used in Stages I and II

System. You are an expert in long-term memory systems for LLM agents. Think carefully, ground every conclusion in the supplied implementation and measurements, and return exactly the requested format.

Task. You are diagnosing a memory system serving many long dialogues between a user and an assistant. The system uses an external LLM to generate multiple MemoryUnits from each dialogue segment. It passes all units from each source session and the raw session text with timestamp to Memory.index in chronological order, building memory incrementally across sessions. For each question, the external LLM generates a typed Query, which is passed to Memory.retrieve once. The retrieved text and AnswerInstruction are then provided to the answer LLM. The goal is to maximize answer accuracy within a 16000-character retrieval budget. Consider the recall-precision trade-off throughout the entire memory pipeline: each memory unit should retain sufficient factual information while avoiding unnecessary noise. Too little retrieval may miss key evidence, while too much may interfere with answering.

The memory system is evaluated across five capabilities:

- `factual_retrieval`: preserve and locate explicit facts, names, exact values, lists, and local details;
- `temporal`: distinguish states of the same fact at different times, order events, determine recency, and track updates over time;
- `preference_extraction`: recover and apply long-term user preferences, instructions, constraints, and needs;
- `multi_session_synthesis`: connect and summarize evidence distributed across multiple sessions;
- `adversarial`: recognize when the dialogue contains no relevant information and abstain, or detect and cautiously handle contradictory or misleading evidence.

Different capabilities may require different memory designs. You can use the Query Planning stage to identify the capabilities required by the question and, when appropriate, adapt the query and retrieval strategy accordingly to preserve accuracy across capabilities.

Stage I: cold start

Analyze the provided measurements and cases carefully. Identify the capability with the largest accuracy gap, diagnose its most important cause in the current implementation, and propose exactly one diagnosis and implementation change to improve the overall accuracy.

Stage II: capability refinement

For the target capability, diagnose the main cause of errors in the current implementation and propose exactly one diagnosis and implementation change to improve the overall accuracy.
Target capability: [Target capability]

Consider both dimensions before choosing that single change:

(A) **Prompt Optimization:** How should MemoryExtractionInstruction, QueryPlanningInstruction, or AnswerInstruction change to guide the task LLM better?

(B) **Memory Design:** How should the MemoryUnit/Query schemas or Memory.index/retrieve logic change to store and retrieve more useful information?

Memory abstractions. *Insert the memory-interface prompt.*

Current implementation. [Complete parent source]

Current measurements. Training accuracy by capability and average score:
[Training capability profile and average score]

Stage I only: evidence-based metrics

Below are diagnostic metrics computed using semantic similarity at different memory stages. The table below reports the average answer accuracy (Acc) and evidence-similarity diagnostics for each capability. Use these metrics to help locate problems in the current implementation. Semantic similarity is robust to paraphrasing, but may remain high even when exact details are missing or changed. These metrics are imperfect intermediate indicators, so analyze them together with representative cases rather than optimizing them directly.

- `evidence_extracted`: how closely golden evidence is represented by generated MemoryUnits from its source segment
- `evidence_oracle_retrieved`: how reachable golden evidence is through Query generation and Memory.retrieve when the evidence itself is used as the question
- `evidence_retrieved`: how much golden evidence Memory.retrieve returns for the real question

[Table with columns: Capability, Acc, `evidence_extracted`, `evidence_oracle_retrieved`, `evidence_retrieved`; one row per capability]

Below are the length-diagnostic measurements across all questions.

- `chars_per_unit`: the average generated MemoryUnit JSON length; larger and smaller values each have recall and precision trade-offs
- `retrieved_chars`: the average real-question retrieval length; larger and smaller values each have recall and precision trade-offs

[Length measurements with name, definition, and current value]

Representative cases. Below are representative cases for reference. “diagnostics” contains the measurement results for each case. “primary” is the primary capability; “secondary” lists the secondary capabilities. “original_text” shows the original raw text of each golden evidence. “extracted_as” shows how golden evidence text is extracted and stored in MemoryUnits by the current implementation. “retrieved” shows text returned by Memory.retrieve for the question.

If “original_text”, “extracted_as”, or “retrieved” is too long, it may be truncated to retain information relevant to answering the question.

[Representative cases serialized as JSON: one per capability in Stage I, or up to five target-capability cases in Stage II]

Previously explored revisions (when available).

Stage I: cold start

Here are previously explored revisions from the same current implementation. You can refer to them and avoid repeating unhelpful revision directions. [Base and child revisions with summaries, modified components, capability scores, and average score]
The following are recent, similar revision directions that did not improve accuracy. Do not repeat these directions. Diagnose a different direction, such as changing another component or another part of the memory pipeline.
[Repeated unsuccessful revisions, when available]

Stage II: capability refinement

Here are previously committed versions for the target capability. Use them to understand which changes have already been explored and avoid reverting useful improvements.
[JSON list with node, summary, components, and training_profile]

Output. Analyze the provided measurements and cases to identify the main reasons why the current implementation fails to produce correct results. Ground the diagnosis in the implementation, measurements, previously explored revisions, repeated nearby unsuccessful directions when shown, and representative cases. Return one diagnosis with one concrete implementation change.

Answer with exactly one JSON object:

```
{
  "diagnosis": "<one code-and-case-grounded problem and
               concrete implementation change>"
}
```

Refinement Coder prompt template used in Stages I and II

System. You are an expert in long-term memory systems for LLM agents. Think carefully, ground every conclusion in the supplied implementation and measurements, and return exactly the requested format.

Task. You are improving a memory system serving many long dialogues between a user and an assistant. The system uses an external LLM to generate multiple MemoryUnits from each dialogue segment. It passes all units from each source session and the raw session text with timestamp to Memory.index in chronological order, building memory incrementally across sessions. For each question, the external LLM generates a typed Query, which is passed to Memory.retrieve once. The retrieved text and AnswerInstruction are then provided to the answer LLM. The goal is to maximize answer accuracy within a 16000-character retrieval budget. Consider the recall-precision trade-off throughout the entire memory pipeline: each memory unit should retain sufficient factual information while avoiding unnecessary noise. Too little retrieval may miss key evidence, while too much may interfere with answering.

The test questions cover temporal reasoning, evidence connected across sessions, facts updated over time, contradictions that must be surfaced, abstention when dialogue contains no answer, and long-term preferences or instructions that should guide an answer.

Implement the requested change based on the diagnosis feedback for the current implementation; do not broaden it into an unrelated redesign. Implement the requested algorithm efficiently without sacrificing its intended functionality or accuracy.

Memory abstractions. *Insert the memory-interface prompt.*

Current implementation. [Complete fixed parent source]

Diagnosis. [Diagnosis]

Representative cases. Use the following cases to understand the diagnosis and implement the requested change. “primary” is the primary capability; “secondary” lists the secondary capabilities. “original_text” shows the original raw text of each golden evidence. “extracted_as” shows how golden evidence text is extracted and stored in MemoryUnits by the current implementation. “retrieved” shows text returned by Memory.retrieve for the question.

If “original_text”, “extracted_as”, or “retrieved” is too long, it may be truncated to retain information relevant to answering the question.

[The same compact cases used for diagnosis, serialized as JSON]

Revision history (when available).

Stage I: cold start

The following are recent, similar revision directions that did not improve accuracy. Do not repeat these directions. Use the diagnosis to make a change in a different component or another part of the memory pipeline.
[Revision summaries and modified components]

Stage II: capability refinement

Here are previously committed versions for the target capability. Use them to understand which changes have already been explored and avoid reverting useful improvements.
[JSON list with node, summary, components, and training_profile]

How to write the patch. The patcher locates hunks from quoted context rather than trusting line numbers. Copy context exactly, begin every changed region with an @@ hunk header, and use separate hunks for non-contiguous edits. The patch must compile, make a real change, and preserve the required schemas, instructions, and three Memory signatures. Return the diff against Current implementation, not against a previous attempt.

Example:

```
@@
-MemoryExtractionInstruction = "Extract the key information
  from the text into several memory units."
+MemoryExtractionInstruction = "Extract distinct facts, updates
  , and preferences from the text into several memory units."
```

Previous-attempt feedback (when available).

If the previous attempt failed during patching, checks, or execution:

Previous failed attempt:

[Previous summary, code, and error]

Repair this error. Return a complete diff against the original Current implementation.

If the previous attempt was measured but not accepted:

Previous measured attempt:

[Previous reply JSON with summary and code]

[Measurement JSON with parent, candidate, and reasons]

Analyze this failure and revise the original parent following the diagnosis.

Output. Answer with exactly one JSON object:

```
{
  "summary": "<one concise change description>",
  "code": "<unified diff against Current implementation>"
}
```

Planning prompt template used in Stage III

System. You are an expert in long-term memory systems for LLM agents. Think carefully, ground every conclusion in the supplied implementation and measurements, and return exactly the requested format.

Task. Plan one integrated Memory Program using Current implementation as the sole base. Compare the supplied capability specialists using their implementations, training gains over the base, and paired cases. Select compatible improvements, retain useful base behavior, and resolve conflicts and shared dependencies. Return an implementation-ready plan: specify what to change, where, and how, including required schema, instruction, indexing, retrieval, and helper updates.

Memory abstractions. *Insert the memory-interface prompt.*

Current implementation. [Complete overall-best base source]

Capability specialist sources. [JSON object identifying base-equivalent versions and distinct specialist sources, with their capability roles and training gains over the base]

Current measurements. Training accuracy by capability and average score:

[Base capability-profile JSON]

Length diagnostics:

- `chars_per_unit`: the average generated MemoryUnit JSON length; larger and smaller values each have recall and precision trade-offs
- `retrieved_chars`: the average real-question retrieval length; larger and smaller values each have recall and precision trade-offs

[JSON list with name, definition, and value for each length diagnostic]

Representative cases. Compare base with capability specialist outputs; `*_parent` denotes a specialist's earlier parent for retention comparison. `"original_text"` shows the original raw text of each golden evidence. `"extracted_as"` shows how golden evidence text is extracted and stored in MemoryUnits by the current implementation. `"retrieved"` shows text returned by `Memory.retrieve` for the question.

If `"original_text"`, `"extracted_as"`, or `"retrieved"` is too long, it may be truncated to retain information relevant to answering the question.

[Paired-case JSON: question, golden_answer, original_text, and the available base, capability_case, and capability_parent outputs; each output contains prediction_answer, answer_score, query_plan, retrieved, and extracted_as]

Output. Return exactly one JSON object:

```
{"Step-by-step plan": "<concrete integration plan specifying changes and dependencies>"}
```

Integration Coder prompt template used in Stage III

System. You are an expert in long-term memory systems for LLM agents. Think carefully, ground every conclusion in the supplied implementation and measurements, and return exactly the requested format.

Task. Implement the integration plan against Current implementation, using the supplied capability specialist sources. Integrate required schema, instruction, indexing, retrieval, and helper dependencies together; resolve conflicts and preserve compatible base behavior. Use measured case changes to refine the integration, not benchmark-specific rules.

Memory abstractions. *Insert the memory-interface prompt.*

Current implementation. [Complete fixed overall-best base source]

Capability specialist sources. [JSON list of distinct specialist sources, their capability_case version names, and training_gain_over_base]

Merge plan. [JSON object containing the integration plan]

Representative cases. `"original_text"` shows the original raw text of each golden evidence. `"extracted_as"` shows how golden evidence text is extracted and stored in MemoryUnits by the current implementation. `"retrieved"` shows text returned by `Memory.retrieve` for the question.

If `"original_text"`, `"extracted_as"`, or `"retrieved"` is too long, it may be truncated to retain information relevant to answering the question.

[Paired-case JSON with the shared question, golden answer, original evidence, and base and capability specialist outputs]

How to write the patch. The patcher locates hunks from quoted context rather than trusting line numbers. Copy context exactly, begin every changed region with an @@ hunk header, and use separate hunks for non-contiguous edits. The patch must compile, make a real change,

and preserve the required schemas, instructions, and three Memory signatures. Return the diff against Current implementation, not against a previous attempt.

Example:

```
@@
-MemoryExtractionInstruction = "Extract the key information
  from the text into several memory units."
+MemoryExtractionInstruction = "Extract distinct facts, updates
  , and preferences from the text into several memory units."
```

Previous attempts (when available). Compare each attempt’s case outputs with the corresponding base and capability specialist outputs. Use the implementations to identify integration gains, conflicts, and regressions. Missing views are unavailable, not unchanged. Return a complete diff against Current implementation.

[JSON object keyed by base_attempt1, base_attempt2, etc.; each entry contains the available source or patch, training result, error, and per-case candidate outputs]

Output. Return exactly one JSON object:

```
{
  "summary": "<one concise change description>",
  "code": "<unified diff against Current implementation>"
}
```

G EVOLVED MEMORY PROGRAM

In this section, we present the final memory programs evolved by PRISMEM on BEAM and Long-MemEval.

G.1 BEAM FINAL MEMORY PROGRAM

The program evolved by PRISMEM on BEAM augments dated atomic memories with topic identifiers and a flag for broadly applicable user preferences or standing instructions.

- Extraction assigns related facts to a shared topic and distinguishes cross-topic preferences from project-specific constraints.
- Indexing stores the embeddings and metadata, recovers missing session dates from the input header, and collects distinct standing preferences separately.
- Planning splits multi-aspect questions into subqueries and adds a query for negative evidence when checking whether an action occurred.
- Each subquery ranks memory units using cosine similarity with weight 0.7 and lexical query-token coverage with weight 0.3, plus a topic-based bonus of 0.05. Within the 16000-character context budget, retrieval reserves space for a standing-preference block capped at 400 characters and divides the remaining space equally among subqueries. It removes exact-text duplicates across subqueries and appends the preference block after the date-prefixed results.
- The answering instruction distinguishes statement time from event time, organizes chronological summaries, separates distinct-event sums from running totals, and applies retrieved preferences to the response. It calls for clarification when affirmative and negative evidence conflict, and abstention when evidence is insufficient.

Final memory programs evolved by PRISMEM for BEAM

```
import re
import numpy as np
from dataclasses import dataclass, field
```

```

MemoryExtractionInstruction = (
    "Extract distinct atomic facts, decisions, preferences, "
    "instructions, and temporal events from the text. "
    "Resolve pronouns and references to full names. Preserve "
    "exact values, names, dates, times, and lists verbatim. "
    "If the text contains a user preference or standing "
    "instruction, capture it explicitly as its own memory "
    "unit. For cross-cutting preferences or standing "
    "instructions that apply broadly regardless of the "
    "specific topic or project (e.g. 'always format code "
    "with syntax highlighting', 'prefer minimal "
    "dependencies', 'always compare multiple solution "
    "methods'), set standing_preference to True. For "
    "preferences tied to a specific project, feature, or "
    "domain (e.g. 'prefers Bootstrap for this website'), set "
    "standing_preference to False. Capture any "
    "contradictions or updates to previously stated facts. "
    "For each fact, assign a concise topic identifier "
    "(entity+thread in lowercase hyphenated form, e.g. "
    "'weather-app-autocomplete', "
    "'greg-research-collaboration', "
    "'triangle-classification-progress') that groups all "
    "related facts about the same ongoing thread, project, "
    "or person across sessions. Set session_date to the "
    "current conversation session date in YYYY-MM-DD format "
    "(the date the user discussed the topic in chat, NOT any "
    "project or schedule date in the content). "
)

QueryPlanningInstruction = (
    "Given the question, decompose it into one or more "
    "retrieval sub-queries. For single-topic questions, "
    "provide exactly one sub-query. For multi-aspect, "
    "synthesis, or multi-part questions, provide one "
    "sub-query per distinct topic or aspect so that all "
    "relevant evidence scattered across sessions is "
    "retrieved independently. Each sub-query should be "
    "concise and contain the key entities, specific time "
    "references, and the particular aspect being asked "
    "about. For temporal questions include relevant date or "
    "event words. For preference or instruction questions "
    "include the domain and the type of constraint. For "
    "existence or experience questions (e.g. 'Have I ever "
    "done X', 'Have I completed X', 'Did I do X before', "
    "'Have I used X'), include an additional sub-query "
    "targeting negation or absence (e.g. 'never X', 'not "
    "done X before', 'no prior experience with X', 'has not "
    "attempted X') so that explicit denial or inexperience "
    "statements are retrieved with equal prominence "
    "alongside affirmative evidence."
)

AnswerInstruction = (
    "Based on the retrieved memory context and the original "
    "question, provide a concise and accurate answer. Each "
    "entry is prefixed with [YYYY-MM-DD] showing the session "
    "date when the user discussed that topic. For questions "
    "about the order or sequence in which the user raised "
    "topics, use these session-date prefixes to determine "
    "chronological order. Distinguish statement time (the "
    "[YYYY-MM-DD] prefix) from event time (dates within the "
    "entry text, e.g. project schedules). For synthesis or "
    "summary questions, group entries by topic and present a "
    "unified chronological narrative of how the user's "

```

```

"project or understanding progressed, describing how "
"earlier sessions set the stage for later developments. "
"For count or total questions about progress across "
"sessions, sum values reported in separate distinct "
"events (e.g. 'completed 10' in one session then "
"'completed 15' in a later session = 25 total); use the "
"latest value only when entries track a single running "
"total. However, if two values appear to be "
"contradictory statements about the same quantity in the "
"same session or context, do NOT sum them-surface the "
"contradiction instead. If the context contains "
"contradictory information, explicitly surface the "
"contradiction and note the conflicting details. If the "
"context does not contain sufficient information to "
"answer, state that clearly rather than guessing. If any "
"retrieved entry states a user preference or standing "
"instruction (e.g., 'always include...', 'I prefer...', "
"'make sure to...'), you MUST apply that constraint to "
"the structure and content of your answer-not merely "
"acknowledge it. If a [Standing Preferences] section "
"appears in the retrieved context, apply every "
"preference listed there. CRITICAL for existence or "
"experience questions (e.g. 'Have I ever X', 'Did I do "
"X', 'Have I completed X before'): if the retrieved "
"context contains BOTH affirmative evidence that the "
"user did, completed, or used X AND explicit statements "
"that the user never did X, has not done X, or has no "
"prior X, your answer MUST lead with the contradiction. "
"Present both the affirmative evidence and the negation "
"statement explicitly, briefly quote or paraphrase each, "
"and ask the user to clarify which is correct. Do NOT "
"resolve the contradiction by picking one side, "
"dismissing one as outdated, or explaining away the "
"other."
)

@dataclass
class MemoryUnit:
    memory_text: str = field(
        metadata={"description": "One atomic piece of information worth remembering: a fact, event, preference, decision, or instruction. Include specific names, values, and dates."}
    )
    session_date: str = field(
        default="",
        metadata={"description": "The conversation session date in YYYY-MM-DD format. This is the date of the current chat session, not any project or schedule date mentioned in the content."}
    )
    topic: str = field(
        default="",
        metadata={"description": "A concise topic identifier in lowercase-hyphenated form (entity+thread) grouping related facts across sessions. Examples: 'weather-app-autocomplete', 'triangle-classification-progress', 'portfolio-website-sprint-1'. Use 'general' if no clear thread."}
    )
    standing_preference: bool = field(
        default=False,
        metadata={"description": "True ONLY for cross-cutting user preferences or standing instructions that apply broadly regardless of topic (e.g. 'always use tree diagrams for probability', '

```

```

        prefer minimal dependencies'). False for topic-specific facts or
        one-off requests."}
    )

@dataclass
class Query:
    query_texts: list[str] = field(
        metadata={"description": "A list of retrieval sub-queries.
        Each sub-query targets one distinct topic or aspect. For single-
        topic questions use one entry. For multi-aspect or synthesis
        questions, decompose into one sub-query per distinct topic so that
        all relevant evidence is retrieved. Each should contain key
        entities, time references, and the specific aspect being asked
        about."}
    )

class Memory:
    def __init__(self, backend):
        self.backend = backend
        self.memories: list[str] = []
        self.embeddings: list[np.ndarray] = []
        self.session_dates: list[str] = []
        self.topics: list[str] = []
        self.standing_prefs: list[str] = []
        self.topic_to_indices: dict[str, list[int]] = {}

    _STOP = frozenset(
        "the a an is are was were in on at to for of and or but if how
        what when where who which did do does have has had i my me you
        your we our they their it its this that with from by as be been
        being am will would can could should may might shall not no yes
        any all each every some there here so than too very just also only
        own same out up down over under again further then once your you
        my me i we they he she it its his her their our your mine yours
        his hers ours theirs".split()
    )

    @staticmethod
    def _parse_session_date(raw_text: str) -> str:
        header = raw_text[:500]
        m = re.search(r'(\d{4})-(\d{2})-(\d{2})', header)
        if m:
            return f"{m.group(1)}-{m.group(2)}-{m.group(3)}"
        m = re.search(r'(\d{1,2})/(\d{1,2})/(\d{4})', header)
        if m:
            return f"{m.group(3)}-{int(m.group(1)):02d}-{int(m.group(2)):02d}"
        return ""

    @staticmethod
    def _content_tokens(text: str) -> set[str]:
        return {t for t in re.findall(r"\b\w+\b", text.lower()) if t
        not in Memory._STOP and len(t) > 1}

    @staticmethod
    def _normalize_topic(topic: str) -> str:
        return re.sub(r'^a-z0-9+', '-', topic.lower()).strip('-') or
        "general"

    def _topic_tokens(self, topic: str) -> set[str]:
        normalized = self._normalize_topic(topic)

```

```

parts = set(normalized.split('-'))
parts.add(normalized)
return {t for t in parts if len(t) > 2}

def index(self, units: list[MemoryUnit], raw_text: str) -> None:
    if not units:
        return
    parsed_date = self._parse_session_date(raw_text)
    texts = [u.memory_text for u in units]
    embs = self.backend.embed(texts)
    for i, u in enumerate(units):
        sd = getattr(u, 'session_date', '') or parsed_date
        topic = getattr(u, 'topic', '') or ''
        sp = getattr(u, 'standing_preference', False)
        self.memories.append(texts[i])
        self.embeddings.append(embs[i])
        self.session_dates.append(sd)
        self.topics.append(topic)
        norm_topic = self._normalize_topic(topic)
        if norm_topic not in self.topic_to_indices:
            self.topic_to_indices[norm_topic] = []
        self.topic_to_indices[norm_topic].append(i)
        if sp and texts[i] not in self.standing_prefs:
            self.standing_prefs.append(texts[i])

def retrieve(self, query: Query) -> str:
    if not self.memories:
        return ""
    sub_queries = query.query_texts
    if not sub_queries:
        return ""

    # Build standing preferences block
    standing_prefs_block = ""
    if self.standing_prefs:
        prefs_lines = [f"- {p}" for p in self.standing_prefs]
        standing_prefs_block = "[Standing Preferences]\n" + "\n".join(prefs_lines)
        if len(standing_prefs_block) > 400:
            standing_prefs_block = standing_prefs_block[:400]
        standing_overhead = (len(standing_prefs_block) + 4) if
standing_prefs_block else 0
    main_budget = OUT_CHAR_BUDGET - standing_overhead
    if main_budget < 800:
        main_budget = OUT_CHAR_BUDGET
        standing_overhead = 0
        standing_prefs_block = ""

    n = len(sub_queries)
    per_query_budget = main_budget // n

    all_embs = np.stack(self.embeddings)
    seen_texts: set[str] = set()
    selected: list[tuple[float, str, str]] = [] # (score,
session_date, text)

    for sq in sub_queries:
        query_emb = self.backend.embed([sq])[0]
        cos_scores = all_embs @ query_emb

        q_tokens = self._content_tokens(sq)
        kw_scores = np.zeros(len(self.memories), dtype=np.float64)

```

```

        if q_tokens:
            for i, txt in enumerate(self.memories):
                t_tokens = self._content_tokens(txt)
                kw_scores[i] = len(q_tokens & t_tokens) / len(
q_tokens)

        # Topic boost
        topic_boost = np.zeros(len(self.memories), dtype=np.
float64)
        if q_tokens:
            for topic, indices in self.topic_to_indices.items():
                t_toks = self._topic_tokens(topic)
                if not t_toks:
                    continue
                overlap = q_tokens & t_toks
                if len(overlap) >= 2:
                    for idx in indices:
                        topic_boost[idx] = 0.05
                elif len(overlap) == 1:
                    for tok in overlap:
                        if len(tok) > 4:
                            for idx in indices:
                                topic_boost[idx] = 0.05
                            break

        scores = 0.7 * cos_scores + 0.3 * kw_scores + topic_boost
        order = np.argsort(-scores)

        used = 0
        for idx in order:
            idx = int(idx)
            txt = self.memories[idx]
            if txt in seen_texts:
                continue
            sd = self.session_dates[idx] if idx < len(self.
session_dates) else ""
            prefix_len = len(sd) + 3 if sd else 0
            cost = len(txt) + prefix_len + 2
            if used + cost > per_query_budget:
                continue
            selected.append((float(scores[idx]), sd, txt))
            seen_texts.add(txt)
            used += cost

        if not selected:
            return standing_prefs_block if standing_prefs_block else "
"

        # Keep relevance order; format with date prefix for temporal
context
        parts: list[str] = []
        total = 0
        for score, sd, txt in selected:
            prefix = f"[{sd}] " if sd else ""
            formatted = prefix + txt
            sep = "\n\n" if parts else ""
            if total + len(sep) + len(formatted) > main_budget:
                break
            parts.append(formatted)
            total += len(sep) + len(formatted)

        result = "\n\n".join(parts)
        if standing_prefs_block:

```

```

        result += "\n\n" + standing_prefs_block
    return result[:OUT_CHAR_BUDGET]

```

G.2 LONGMEMEVAL FINAL MEMORY PROGRAM

The program evolved by PRISMEM on LongMemEval uses a flat dense index of dated atomic memories.

- Extraction preserves exact names, values, dates, lists, and user preferences, resolves references to explicit entity names, and records the source session date.
- Indexing stores each unit's text, normalized embedding, and date in aligned lists.
- Planning generates a single query tailored to the question's entities, factual qualifiers, time references, or preference cues.
- Retrieval ranks all memory units by cosine similarity, keeps those scoring at least 0.25, and packs date-prefixed units in relevance order within a 16000-character context budget.
- The answering instruction applies remembered constraints to recommendations, uses date prefixes for temporal calculations, enforces factual qualifiers, aggregates distinct occurrences, and requests abstention when a specific fact is unsupported.

Final memory programs evolved by PRISMEM for LongMemEval

```

from dataclasses import dataclass, field
import numpy as np

MemoryExtractionInstruction = (
    "Extract distinct atomic facts, procedures, "
    "relationships, decisions, and preferences from the text "
    "into several memory units. Resolve pronouns and "
    "references to explicit entity names. Preserve exact "
    "names, numeric values, dates/times, and lists verbatim. "
    "For each unit, also set session_date to the ISO date "
    "(YYYY-MM-DD) of the conversation, taken from the "
    "timestamp in the conversation header."
)

QueryPlanningInstruction = (
    "First, classify the question into one of these types: "
    "(A) preference/suggestion/advice/recommendation, (B) "
    "temporal/ordering/sequence, (C) factual retrieval with "
    "specific qualifiers, (D) multi-session synthesis or "
    "aggregate/count, (E) verification or potential "
    "abstention. Then write query_text tailored to that "
    "type. (A) For preference/suggestion/advice questions, "
    "do NOT write only a surface-topic query (e.g., "
    "'activities to do during commute'). Instead, construct "
    "a query that explicitly targets the user's stated "
    "preferences, interests, exclusions, constraints, and "
    "relevant personal history in the domain. Combine domain "
    "keywords from the question (e.g., 'commute', 'podcast', "
    "'listening', 'hotel') with preference-signal terms "
    "(e.g., 'user prefers', 'interested in', 'wants to "
    "explore', 'branch out', 'does not want', 'exclusion', "
    "'genre', 'specifically likes', 'personal history') so "
    "that memory units recording what the user likes, "
    "dislikes, wants to try next, and their background in "
    "that domain rank highest. (B) For temporal/ordering "
    "questions, include the specific entities and any time "
    "references so dated events are retrieved. (C) For "

```

```

"factual retrieval, include exact entity names and ALL "
"qualifying criteria from the question. (D) For "
"multi-session synthesis or aggregate questions, name "
"each distinct component or entity mentioned so evidence "
"across sessions is retrieved. (E) For verification or "
"abstention questions, include the specific role, title, "
"or condition named in the question to confirm whether "
"it exists. Keep the query concise but information-dense."
)
AnswerInstruction = (
"First, identify the question type and follow the "
"corresponding strategy. Then answer using ONLY the "
"retrieved memory text.\n\n=== RECOMMENDATION / "
"SUGGESTION / ADVICE / OPINION QUESTIONS (HIGHEST "
"PRIORITY) ===\nIf the question asks for a "
"recommendation, suggestion, advice, or opinion (e.g., "
"'recommend resources', 'what should I try', 'should I "
"attend X', 'what would be good for me', 'do you think "
"it would be a good idea', 'what should I do', 'any tips "
"or ideas'):\n- The user's stated preferences, "
"interests, skill level, software/tools they use, "
"exclusions, and personal history found in the retrieved "
"memory ARE the answer. You MUST generate a concrete "
"recommendation or suggestion explicitly shaped by those "
"preferences.\n- It is NEVER correct to answer "
"'information not available', 'no specific resources "
"found', or 'the specific information is not available' "
"when the retrieved memory contains ANY domain "
"preference, interest, tool, or relevant personal "
"context for the user. The preference itself IS the "
"useful information. Do not treat the user's preferences "
"as 'similar but different facts' to be rejected.\n- "
"Example: If asked to recommend video editing learning "
"resources and the memory says the user enjoys Adobe "
"Premiere Pro and wants to learn advanced settings, the "
"answer must recommend Adobe Premiere Pro advanced "
"tutorials or resources, NOT say 'no specific resources "
"are available.'\n- Enumerate every explicitly stated "
"preference, constraint, exclusion, and relevant "
"personal fact from the retrieved memory. A preference "
"stored without an explicit location, date, or occasion "
"scope is a GENERAL preference that applies to any "
"suggestion in the same domain. Apply all as binding "
"constraints. Generate suggestions that respect all "
"listed constraints, explicitly note exclusions, and "
"reference the user's personal history as supporting "
"evidence. Never ignore a stated exclusion.\n\n=== "
"TEMPORAL / ORDERING / DURATION / 'HOW MANY DAYS OR "
"WEEKS' QUESTIONS ===\n- The [YYYY-MM-DD] prefix on each "
"memory line IS the date of the event or conversation "
"that produced that memory. Treat it as the event date "
"for that fact. Use these dates to compute differences.\n"
"- When the question asks for a duration, difference, or "
"ordering between two or more events, identify the "
"[YYYY-MM-DD] date for each event from the retrieved "
"memory, then compute the exact numerical difference in "
"days (or convert to weeks if the question asks for "
"weeks). Show the calculation briefly.\n- Example: If "
"one event is dated [2023-03-17] and another is dated "
"[2023-04-23], the difference is 37 days. State the "
"exact number.\n- For ordering questions, sort all dated "
"events chronologically by their [YYYY-MM-DD] prefix and "
"present the ordered list using exact entity names as "

```

```

"written in the memory.\n\n=== FACTUAL RETRIEVAL WITH "
"SPECIFIC QUALIFIERS ===\n- When the question contains "
"multiple qualifying criteria, select the single memory "
"fact that satisfies ALL of them. Do not pick a "
"partially matching fact.\n- Exactness: Reproduce entity "
"names, dates, times, and numeric values exactly as they "
"appear in the retrieved memory. Never paraphrase or "
"approximate. For example, use 'Modern Art Museum' not "
"'Modern Art Gallery'.\n- Near-miss/abstention: If the "
"retrieved memory mentions a similar but different fact "
"than what the question specifically asks about, do NOT "
"substitute it. State that the specific information is "
"not available. For instance, if asked about an "
"'undergrad course research project' poster but the "
"memory only records a 'thesis research' poster, answer "
"that the specific information is not available. This "
"rule does NOT apply to recommendation or suggestion "
"questions covered above.\n\n=== MULTI-SESSION SYNTHESIS "
"/ AGGREGATE / COUNT / SUM QUESTIONS ===\n- Identify "
"EVERY relevant numeric value or duration across all "
"retrieved memory units and sum or count them to produce "
"a single final answer.\n- Include values that refer to "
"the same entity but represent distinct occurrences "
"(e.g., the same game played on two different "
"difficulties counts as two separate values).\n- "
"Example: If asked for the total weeks to complete two "
"activities, and memory shows 2 weeks for one and 1.5 "
"weeks for the other, the answer is 3.5 weeks, not two "
"separate answers.\n\n=== VERIFICATION / ABSTENTION "
"QUESTIONS ===\n- Include the specific role, title, or "
"condition named in the question to confirm whether it "
"exists in the retrieved memory.\n- If the retrieved "
"memory does not contain the specific information asked "
"about, state clearly that it is not available. Do not "
"substitute a similar fact.\n\n=== GENERAL RULES (apply "
"to all types) ===\n- Keep the answer concise. Do not "
"add information not present in the retrieved memory.\n- "
"Use exact entity names, dates, and values as written in "
"the retrieved memory."
)

@dataclass
class MemoryUnit:
    memory_text: str = field(
        metadata={"description": "One piece of information worth remembering from the text"}
    )
    session_date: str = field(
        default="",
        metadata={"description": "The ISO date (YYYY-MM-DD) of the session from which this memory was extracted, taken from the conversation timestamp"}
    )

@dataclass
class Query:
    query_text: str = field(
        metadata={"description": "A query for retrieving information relevant to the question"}
    )

```

```

class Memory:
    def __init__(self, backend):
        self.backend = backend
        self.memories: list[str] = []
        self.embeddings: list[np.ndarray] = []
        self.session_dates: list[str] = []

    def index(self, units: list[MemoryUnit], raw_text: str) -> None:
        texts = [unit.memory_text for unit in units]
        dates = [unit.session_date for unit in units]
        if texts:
            vecs = self.backend.embed(texts)
            self.memories.extend(texts)
            self.session_dates.extend(dates)
            self.embeddings.extend(list(vecs))

    def retrieve(self, query: Query) -> str:
        if not self.memories:
            return ""
        q_vec = self.backend.embed([query.query_text])[0]
        all_vecs = np.stack(self.embeddings)
        scores = all_vecs @ q_vec
        sorted_indices = np.argsort(-scores)
        selected: list[str] = []
        total_chars = 0
        MIN_SIMILARITY = 0.25
        for idx in sorted_indices:
            if scores[idx] < MIN_SIMILARITY:
                break
            text = self.memories[idx]
            date = self.session_dates[idx]
            tagged = f"[{date}] {text}" if date else text
            extra = len(tagged) + (2 if selected else 0)
            if total_chars + extra > OUT_CHAR_BUDGET:
                continue
            selected.append(tagged)
            total_chars += extra
        return "\n\n".join(selected)

```